Python Activity Based Learning

1.Energy Conservation Before-and-After Analysis

1. Problem Statement

Energy conservation is important for reducing electricity consumption and saving energy. This project compares electricity consumption before and after implementing energy-saving methods. It calculates the total energy saved and the percentage reduction in electricity consumption. It also identifies whether energy consumption has increased or decreased.

2. Objectives

- To calculate the difference in electricity consumption before and after conservation.

- To calculate the amount of energy saved in kilowatt-hours (kWh).

- To calculate the percentage reduction in energy consumption.

- To identify whether electricity consumption has increased, decreased, or remained the same.

- To understand the use of conditional statements and arithmetic operations in Python.

3. Assumptions

1. Electricity consumption before conservation must be greater than zero.

2. Electricity consumption after conservation must be zero or greater.

3. Both input values are measured in kilowatt-hours (kWh).

4. The entered values are numerical.

5. The program compares the two values to determine the result.

4. Algorithm

1. Start the program.

2. Display the title of the energy conservation analysis.

3. Read electricity consumption before conservation.

4. Read electricity consumption after conservation.

5. Check whether the input values are valid.

6. If the values are invalid, display an error message.

7. Otherwise, compare the before and after consumption.

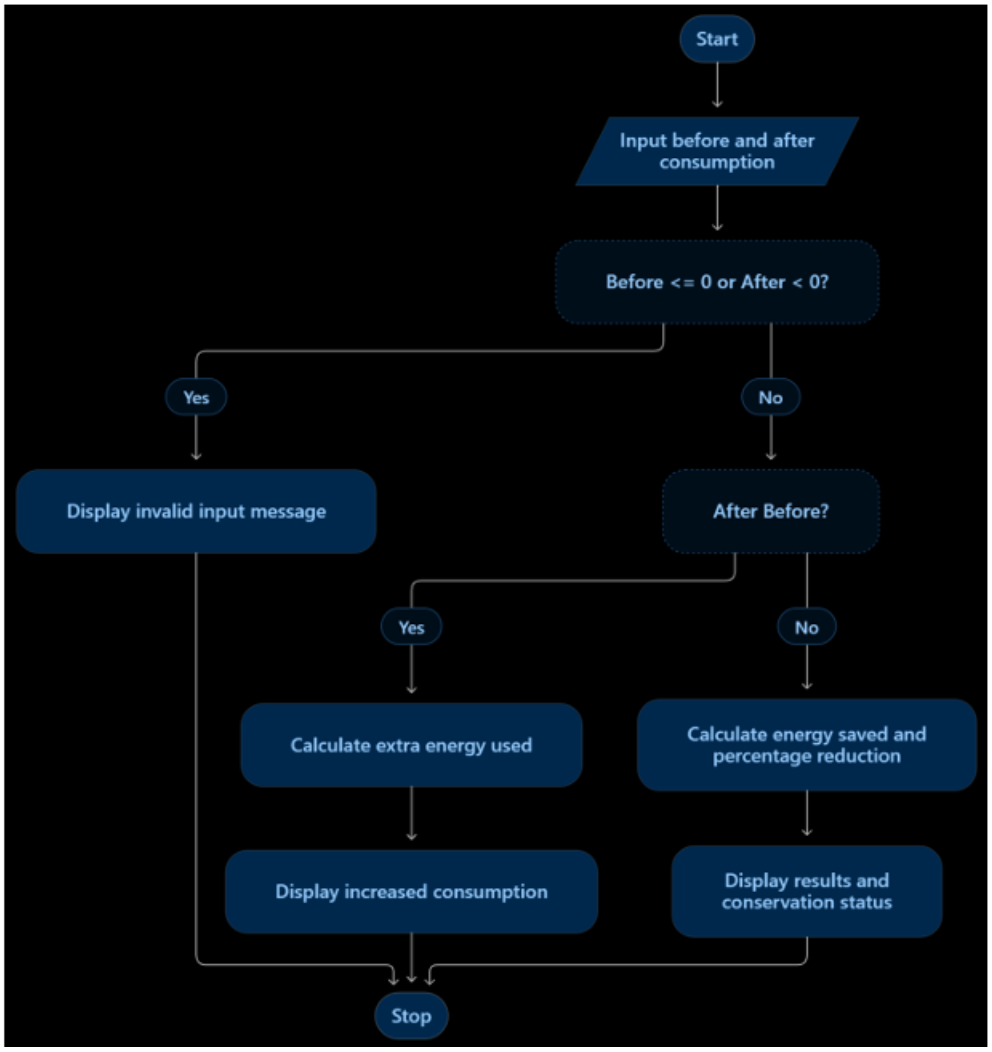
8. If the after-consumption value is greater, display the increase in energy usage.

9. Otherwise, calculate the energy saved and percentage reduction.

10. Display the results and conservation status.

11. Stop the program.

5. Flowchart



6. Pseudocode

BEGIN

DISPLAY "ENERGY CONSERVATION ANALYSIS"

```
INPUT before
INPUT after

IF before <= 0 OR after < 0 THEN
   DISPLAY "Please enter valid values"

ELSE IF after > before THEN
   extra = after - before
   DISPLAY "Electricity consumption has increased"
   DISPLAY extra

ELSE
   saved = before - after
   percentage = (saved / before) * 100

   DISPLAY before
   DISPLAY after
   DISPLAY saved
   DISPLAY percentage

   IF saved > 0 THEN
      DISPLAY "Energy conservation was successful"
   ELSE
      DISPLAY "No energy was saved"
   END IF

END IF
```

END

7. Python Source Code

```python
#Energy Conservation Before-and-After Analysis

print("ENERGY CONSERVATION ANALYSIS")

before = float(input("Enter electricity consumption before conservation (kWh): "))
after = float(input("Enter electricity consumption after conservation (kWh): "))

if before <= 0 or after < 0:
    print("Please enter valid electricity consumption values.")
elif after > before:
    print("Electricity consumption has increased.")
    print("Extra energy used:", after - before, "kWh")
else:
    saved = before - after
    percentage = (saved / before) * 100

    print("\n--- Results ---")
    print("Consumption before:", before, "kWh")
    print("Consumption after:", after, "kWh")
    print("Energy saved:", saved, "kWh")
    print("Energy reduction:", round(percentage, 2), "%")

    if saved > 0:
        print("Result: Energy conservation was successful!")
    else:
        print("Result: No energy was saved.")
```

8. Test Cases

Test case	Before (kWh)	After (kWh)	Expected result
1	500	350	150 kWh saved; 30% reduction
2	400	300	100 kWh saved; 25% reduction
3	250	250	No energy saved
4	300	400	Consumption increased by 100 kWh
5	0	100	Invalid input
6	500	-20	Invalid input

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe
sers/Velag/OneDrive/Desktop/Python/Activity1.py
Enter electricity consumption after conservation (kWh): 350

--- Results ---
Consumption before: 500.0 kWh
Consumption after: 350.0 kWh
Energy saved: 150.0 kWh
Energy reduction: 30.0 %
Result: Energy conservation was successful!
PS C:\Users\Velag\OneDrive\Desktop\Python>
```

10. Conclusion

The Energy Conservation Before-and-After Analysis program compares electricity consumption before and after energy-saving measures. It calculates the amount of energy saved and the percentage reduction in consumption. It also detects increased consumption and invalid inputs. Through this project, we learn how to use Python input and output statements, variables, arithmetic operators, and conditional statements to solve a real-life problem.

2. Bus Passenger Occupancy Simulation

1. Problem Statement

Bus Passenger Occupancy Simulation is a Python program used to calculate the number of passengers inside a bus after passengers board and get off at a bus stop. The program checks the bus capacity, calculates the remaining passengers, and determines the available seats. It also checks whether the number of passengers exceeds the bus capacity.

2. Objectives

- To calculate the number of passengers inside a bus.

- To simulate passengers boarding and getting off.

- To check whether the bus exceeds its maximum capacity.

- To calculate the number of available seats.

- To understand the use of variables, arithmetic operations, and conditional statements in Python.

3. Assumptions

1. The bus has a fixed seating capacity.

2. All passenger counts are entered as whole numbers.

3. The initial passenger count cannot exceed the bus capacity.

4. The number of passengers getting off cannot exceed the initial passenger count.

5. The program processes passenger boarding and getting off at one bus stop.

6. The program does not allow the final passenger count to exceed the bus capacity.

4. Algorithm

1. Start the program.

2. Enter the maximum capacity of the bus.

3. Enter the current number of passengers.

4. Enter the number of passengers getting off.

5. Enter the number of passengers boarding.

6. Check whether the inputs are valid.

7. If any value is invalid, display an error message.

8. Calculate the passengers remaining after some passengers get off.

9. Calculate the available seats.

10. Check whether the boarding passengers exceed the available seats.

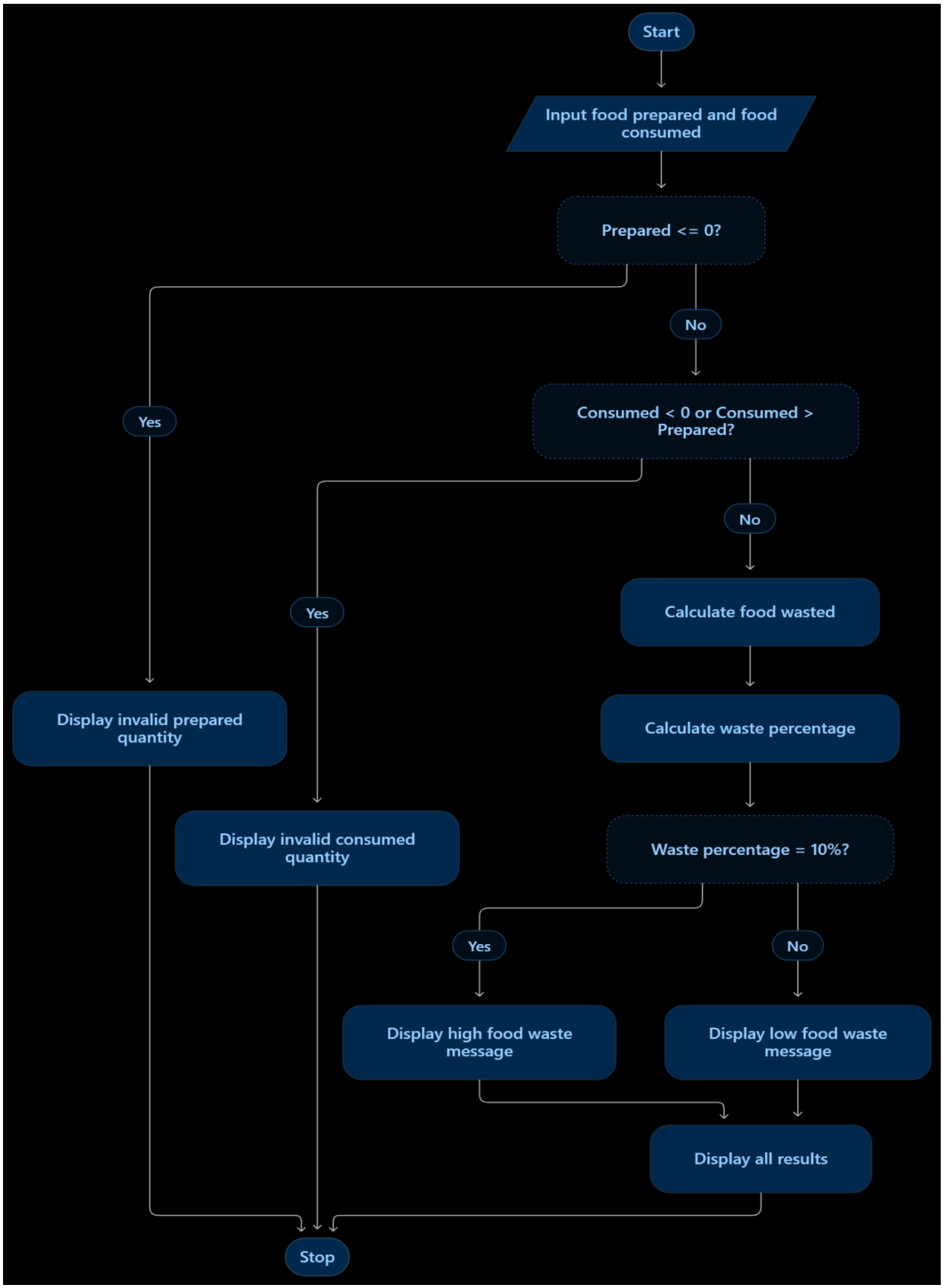
11. If they exceed the available seats, display the maximum number of passengers who can board.

12. Otherwise, calculate and display the final passenger count and available seats.

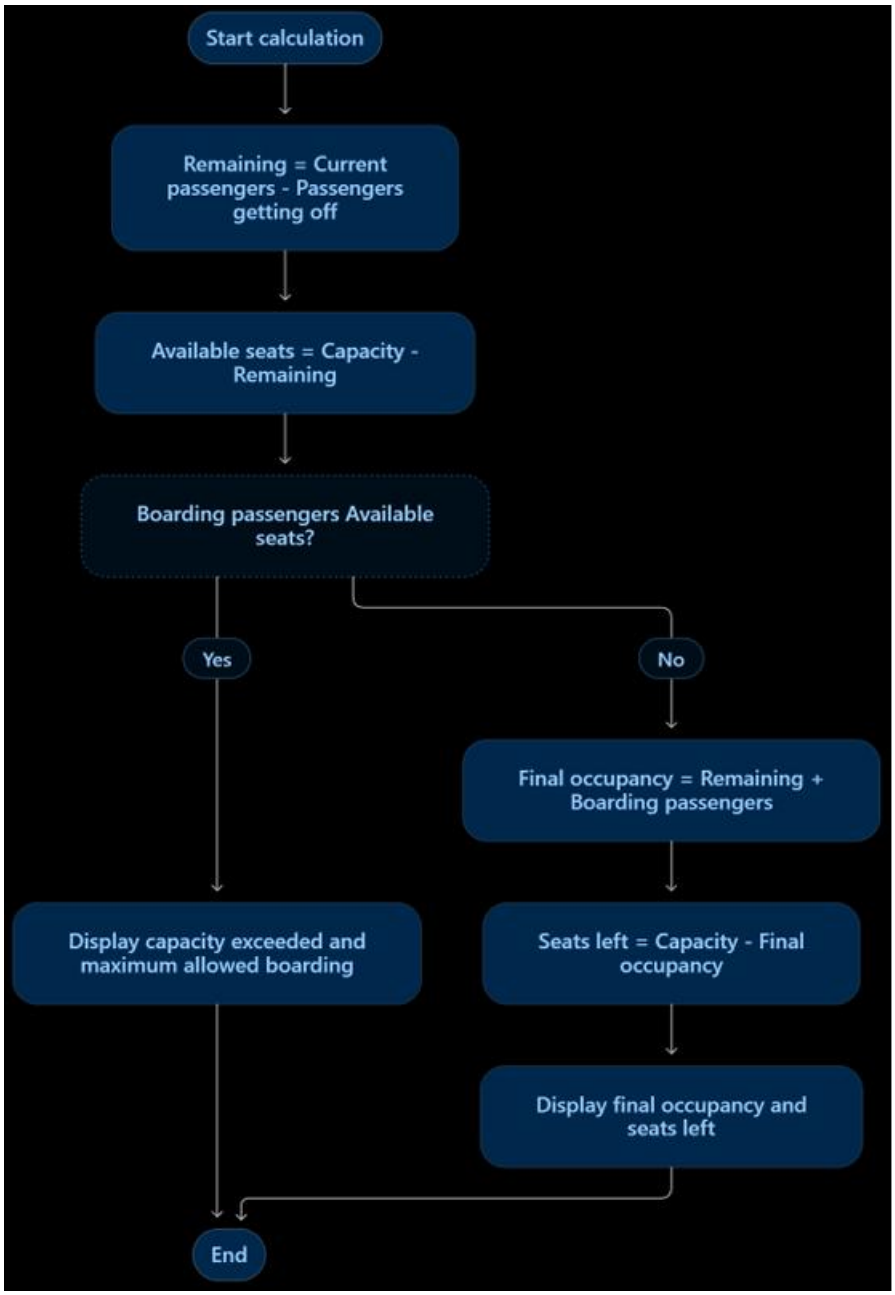
13. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Passenger occupancy calculation



6. Pseudocode

BEGIN

INPUT capacity

INPUT current_passengers

INPUT getting_off

INPUT boarding

```
IF capacity <= 0 OR current_passengers < 0
  OR current_passengers > capacity
  OR getting_off < 0
  OR boarding < 0 THEN

   DISPLAY "Invalid input"

ELSE IF getting_off > current_passengers THEN

   DISPLAY "Invalid number of passengers getting off"

ELSE

   remaining = current_passengers - getting_off
   available = capacity - remaining

   IF boarding > available THEN
      DISPLAY "Bus capacity exceeded"
      DISPLAY "Maximum passengers allowed to board:", available
   ELSE
      final = remaining + boarding
      seats = capacity - final

      DISPLAY remaining
      DISPLAY final
      DISPLAY seats
   END IF
```

END IF

END

7. Python Source Code

```
# Bus Passenger Occupancy Simulation

print("BUS PASSENGER OCCUPANCY SIMULATION")

capacity = int(input("Enter bus capacity: "))
current = int(input("Enter current passengers: "))
getting_off = int(input("Enter passengers getting off: "))
boarding = int(input("Enter passengers boarding: "))

if (capacity <= 0 or current < 0 or current > capacity
        or getting_off < 0 or boarding < 0):
    print("Invalid input")

elif getting_off > current:
    print("Invalid number of passengers getting off")

else:
    remaining = current - getting_off
    available = capacity - remaining
```

```

    print("\nPassengers after getting off:", remaining)
    print("Available seats:", available)

    if boarding > available:
        print("Bus capacity exceeded!")
        print("Maximum passengers allowed to board:", available)
    else:
        final = remaining + boarding
        seats = capacity - final

        print("Passengers after boarding:", final)
        print("Seats remaining:", seats)
        print("Bus occupancy simulation completed.")
```

8. Test Cases

| Test case | Capacity | Current | Getting off | Boarding | Expected result |

| --- | --- | --- | --- | --- | --- |

| 1 | 50 | 30 | 5 | 10 | Final passengers: 35; seats: 15 |

| 2 | 40 | 25 | 10 | 20 | Final passengers: 35; seats: 5 |

| 3 | 30 | 25 | 5 | 15 | Capacity exceeded; maximum boarding: 10 |

| 4 | 50 | 20 | 25 | 5 | Invalid number of passengers getting off |

| 5 | 40 | 45 | 5 | 2 | Invalid input |

| 6 | 50 | 50 | 10 | 10 | Final passengers: 50; seats: 0 |

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe
sers/Velag/OneDrive/Desktop/Python/Activity2.py
Enter passengers getting off: 5
Enter passengers boarding: 10

Passengers after getting off: 25
Available seats: 25
Passengers after boarding: 35
Seats remaining: 15
Bus occupancy simulation completed.
PS C:\Users\Velag\OneDrive\Desktop\Python>
```

10. Conclusion

The Bus Passenger Occupancy Simulation program calculates the number of passengers inside a bus after boarding and getting off. It checks the bus capacity, identifies invalid passenger counts, and calculates the number of available seats. This project helps us understand basic Python concepts such as variables, arithmetic operations, input and output statements, and conditional statements. It also demonstrates how Python can be used to solve simple real-life transportation problems.

3. Food Waste Monitoring System

1. Problem Statement

Food waste is a common problem in homes, hostels, college canteens, and restaurants. A large amount of food is wasted when more food is prepared than required. The Food Waste Monitoring System is a simple Python program that calculates the amount of food wasted by comparing the quantity of food prepared with the quantity consumed. It also calculates the waste percentage and displays a message to help monitor food waste.

2. Objectives

- To record the quantity of food prepared and consumed.

- To calculate the total quantity of food wasted.

- To calculate the percentage of food wasted.

- To identify whether food waste is low or high.

- To encourage better food management and reduce food wastage.

- To understand Python variables, arithmetic operations, and conditional statements.

3. Assumptions

1. Food quantities are measured in kilograms (kg).

2. The quantity of food prepared must be greater than zero.

3. The quantity consumed cannot be negative.

4. The quantity consumed cannot exceed the quantity prepared.

5. The program monitors food waste for one day or one meal.

6. For this program, waste below 10% is considered low, while waste of 10% or more is considered high. This is a sample threshold used for the project.

4. Algorithm

1. Start the program.

2. Display the title of the Food Waste Monitoring System.

3. Enter the quantity of food prepared in kilograms.

4. Enter the quantity of food consumed in kilograms.

5. Check whether the input values are valid.

6. If the prepared quantity is zero or negative, display an invalid input message.

7. If the consumed quantity is negative or greater than the prepared quantity, display an error message.

8. Otherwise, calculate the quantity of food wasted.

9. Calculate the percentage of food wasted.

10. If the waste percentage is 10% or more, display a high food waste message.

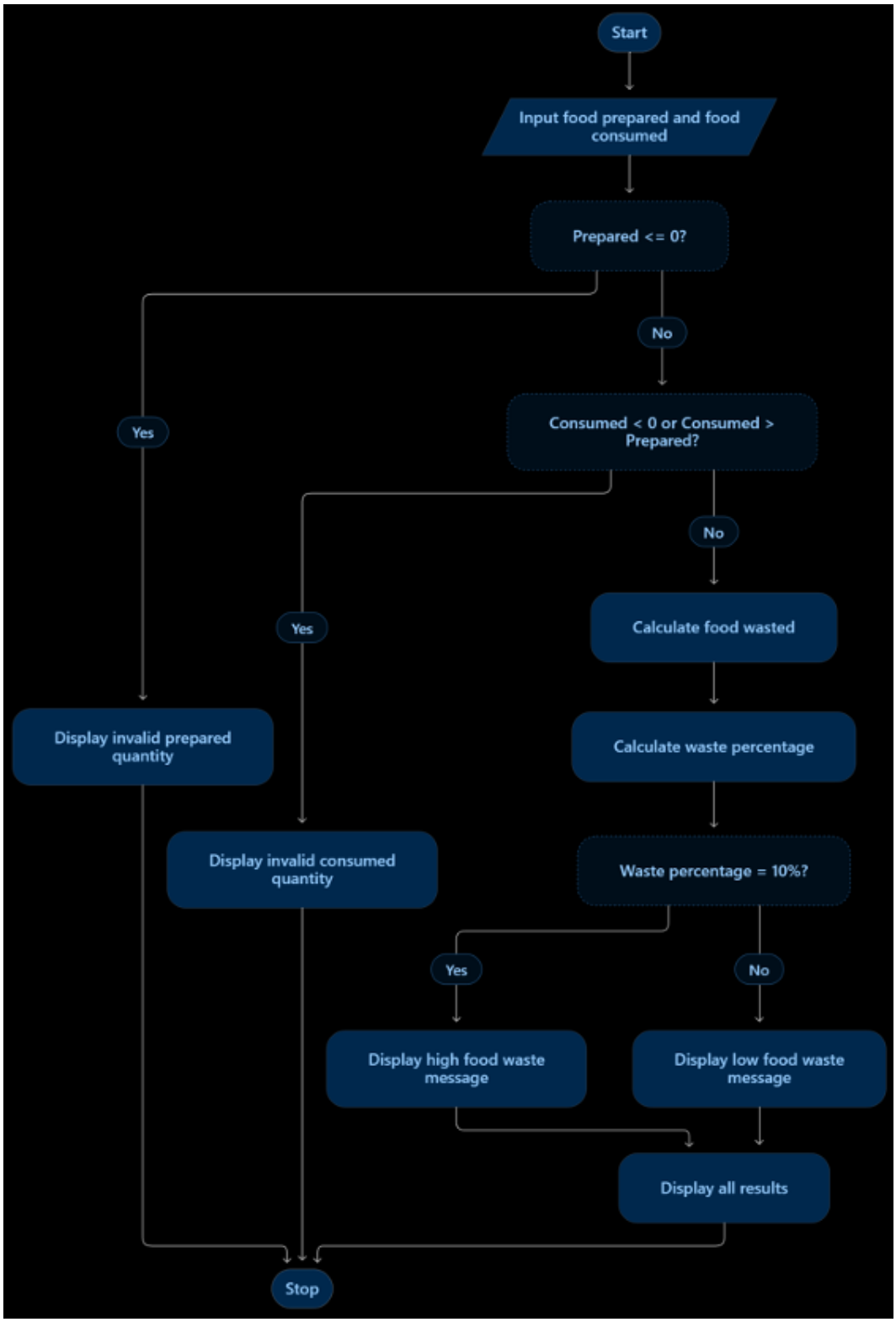
11. Otherwise, display a low food waste message.

12. Display the food prepared, consumed, wasted, and waste percentage.

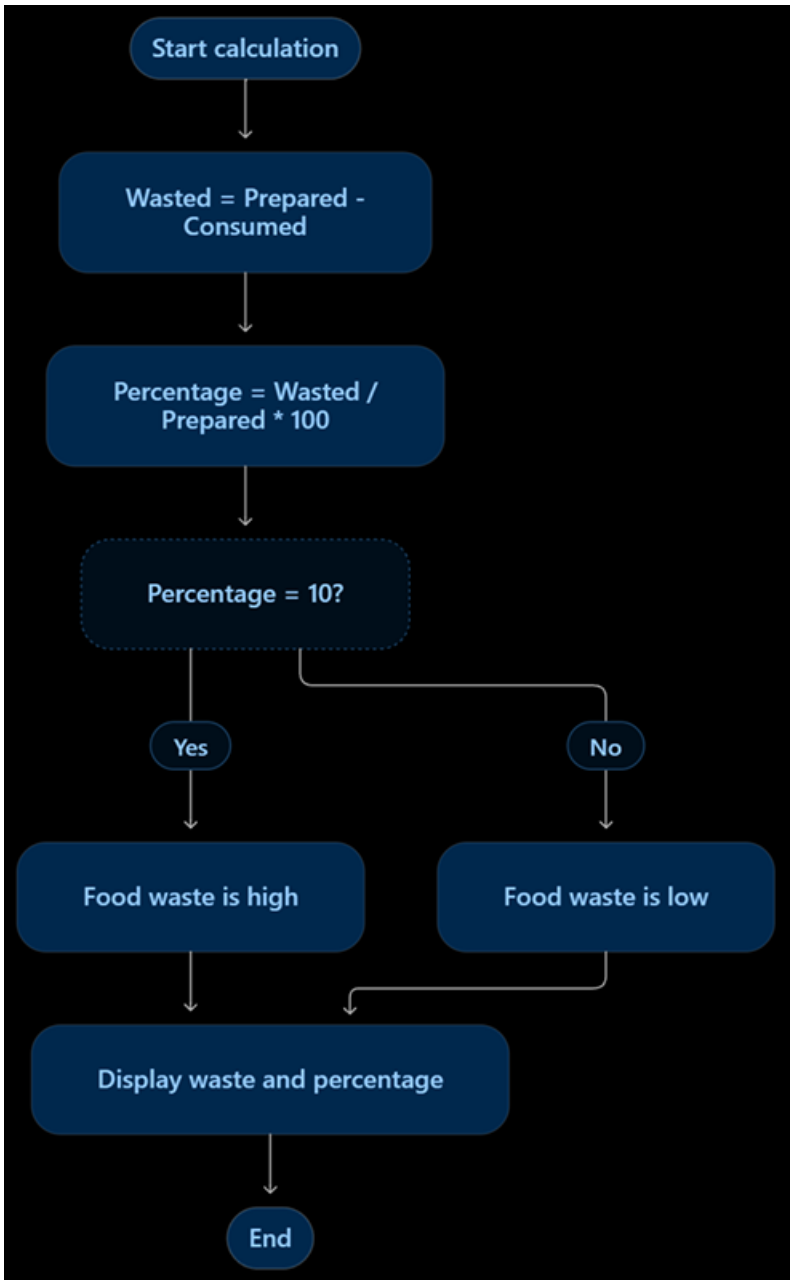
13. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Food waste calculation



6. Pseudocode

BEGIN

DISPLAY "FOOD WASTE MONITORING SYSTEM"

INPUT prepared

INPUT consumed

```
  IF prepared <= 0 THEN
    DISPLAY "Invalid prepared quantity"

  ELSE IF consumed < 0 OR consumed > prepared THEN
    DISPLAY "Invalid consumed quantity"

  ELSE
    wasted = prepared - consumed
    percentage = (wasted / prepared) * 100

    DISPLAY prepared
    DISPLAY consumed
    DISPLAY wasted
    DISPLAY percentage

    IF percentage >= 10 THEN
      DISPLAY "Food waste is high"
    ELSE
      DISPLAY "Food waste is low"
    END IF

  END IF

END
```

7. Python Source Code

```python
# Food Waste Monitoring System

print("FOOD WASTE MONITORING SYSTEM")

prepared = float(input("Enter food prepared (kg): "))
consumed = float(input("Enter food consumed (kg): "))

if prepared <= 0:
    print("Invalid prepared quantity")

elif consumed < 0 or consumed > prepared:
    print("Invalid consumed quantity")

else:
    wasted = prepared - consumed
    percentage = (wasted / prepared) * 100
```

```python
print("\n--- Food Waste Report ---")
print("Food prepared:", prepared, "kg")
print("Food consumed:", consumed, "kg")
print("Food wasted:", wasted, "kg")
print("Waste percentage:", round(percentage, 2), "%")

if percentage >= 10:
    print("Status: Food waste is high.")
else:
    print("Status: Food waste is low.")
```

8. Test Cases

Test case	Prepared (kg)	Consumed (kg)	Expected result
1	100	90	10 kg wasted; 10% waste — High
2	50	48	2 kg wasted; 4% waste — Low
3	80	60	20 kg wasted; 25% waste — High
4	40	40	0 kg wasted; 0% waste — Low
5	0	0	Invalid prepared quantity
6	50	60	Invalid consumed quantity

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe
sers/Velag/OneDrive/Desktop/Python/Activity3.py
Enter food prepared (kg): 100
Enter food consumed (kg): 90

--- Food Waste Report ---
Food prepared: 100.0 kg
Food consumed: 90.0 kg
Food wasted: 10.0 kg
Waste percentage: 10.0 %
Status: Food waste is high.
```

10. Applications

- College canteens: To monitor the quantity of food wasted daily.

- Hostels: To compare food prepared with food consumed.

- Restaurants: To understand food wastage and improve food planning.

- Homes: To encourage better meal planning.

- Food management: To identify high food waste and promote conservation.

11. Conclusion

The Food Waste Monitoring System is a simple Python project that calculates the quantity and percentage of food wasted. It checks input values and classifies food waste as low or high based on the selected threshold. This project helps students understand variables, arithmetic operators, input and output statements, and conditional statements in Python. It also demonstrates how programming can help address a real-life problem and encourage responsible food management.

4.Multi-Product Price Fluctuation Analysis

1. Problem Statement

The prices of products may change over time due to demand, supply, and market conditions. Tracking these changes helps customers and shopkeepers understand price fluctuations. The Multi-Product Price Fluctuation Analysis system is a Python program that accepts the previous and current prices of multiple products. It calculates the price difference and percentage change for each product and displays whether its price has increased, decreased, or remained unchanged.

2. Objectives

- To store the previous and current prices of multiple products.

- To calculate the difference between product prices.

- To calculate the percentage change in price.

- To identify price increases, decreases, and unchanged prices.

- To understand the use of loops, lists, arithmetic operations, and conditional statements in Python.

3. Assumptions

1. The user enters the number of products to be analyzed.

2. Product prices are entered in the same currency, such as Indian rupees (₹).

3. Previous prices must be greater than zero.

4. Current prices must be zero or greater.

5. Each product is analyzed independently.

6. A price change is calculated by comparing the current price with the previous price.

4. Algorithm

1. Start the program.

2. Display the title of the analysis.

3. Enter the number of products to be analyzed.

4. Check whether the number of products is greater than zero.

5. If the number is invalid, display an error message.

6. For each product, enter its name, previous price, and current price.

7. Check whether the previous and current prices are valid.

8. If the prices are invalid, display an error message and move to the next product.

9. Calculate the price difference by subtracting the previous price from the current price.

10. Calculate the percentage change in price.

11. If the price difference is positive, display a price increase.

12. If the price difference is negative, display a price decrease.

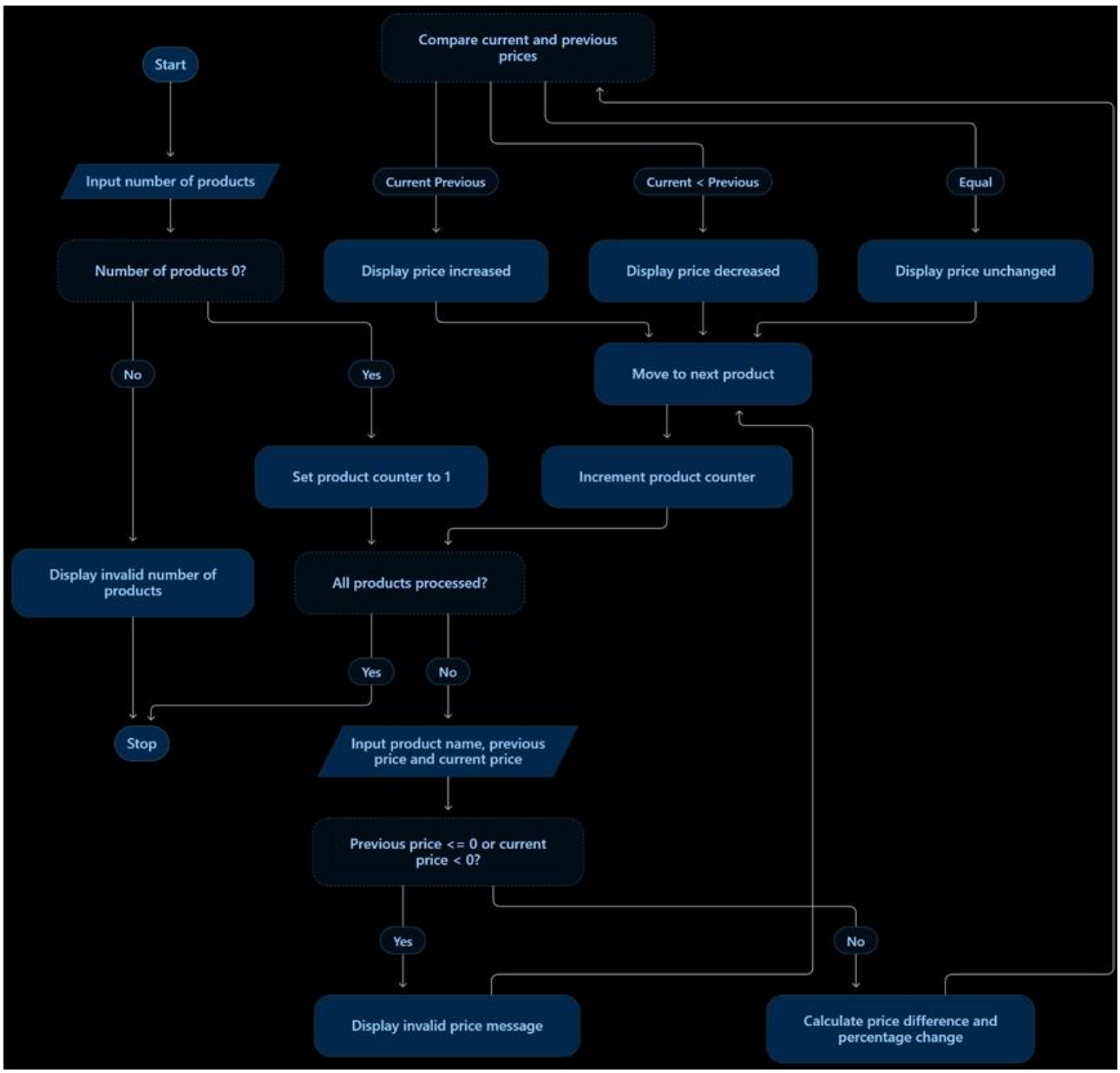
13. If the price difference is zero, display that the price is unchanged.

14. Repeat the process for all products.

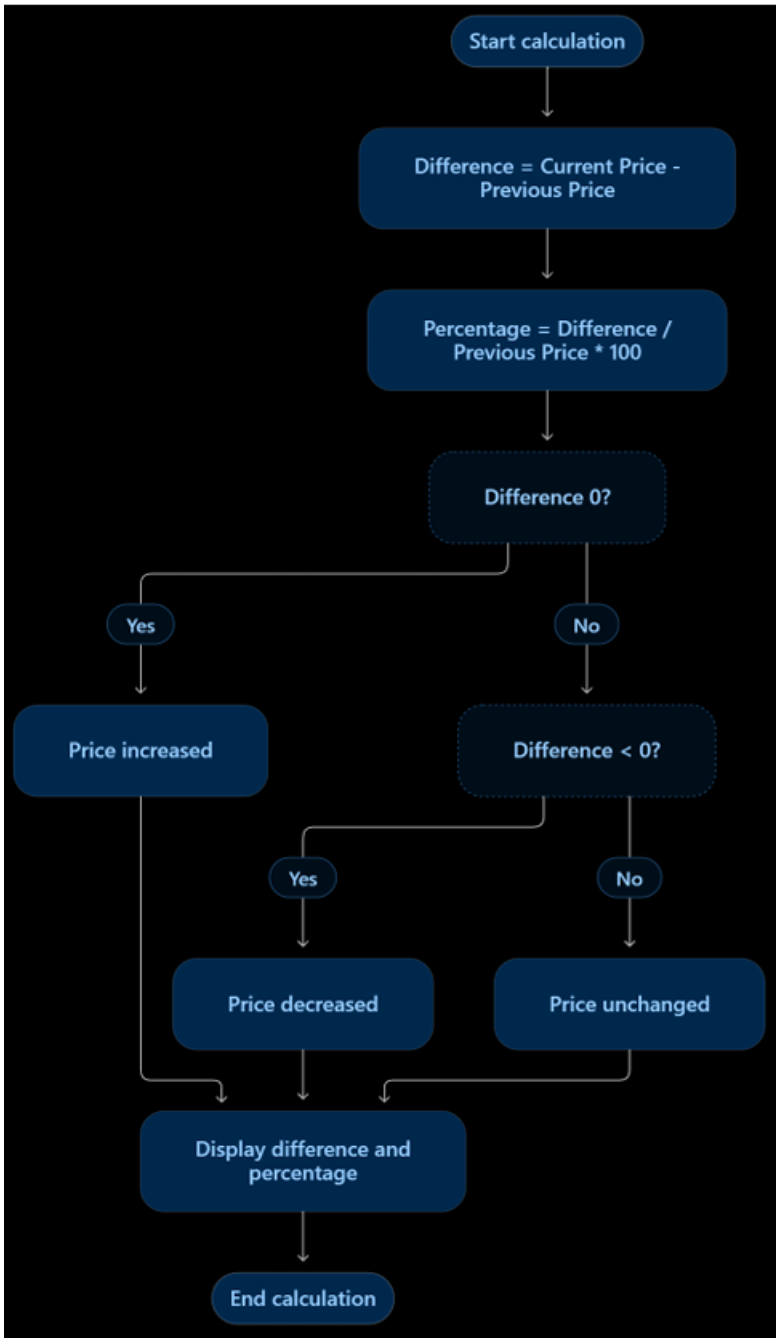
15. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Price fluctuation calculation



6. Pseudocode

BEGIN

DISPLAY "MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS"

INPUT n

```
IF n <= 0 THEN
   DISPLAY "Invalid number of products"

ELSE
   FOR each product from 1 to n

      INPUT product_name
      INPUT previous_price
      INPUT current_price

      IF previous_price <= 0 OR current_price < 0 THEN
         DISPLAY "Invalid price"

      ELSE
         difference = current_price - previous_price
         percentage = (difference / previous_price) * 100

         DISPLAY product_name
         DISPLAY difference
         DISPLAY percentage

         IF difference > 0 THEN
            DISPLAY "Price increased"
         ELSE IF difference < 0 THEN
            DISPLAY "Price decreased"
         ELSE
            DISPLAY "Price unchanged"
```

END IF

END IF

END FOR

END IF

END

7. Python Source Code

```
print("MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS")

n = int(input("Enter the number of products: "))

if n <= 0:
    print("Invalid number of products")

else:
    for i in range(n):
        print("\nProduct", i + 1)

        name = input("Enter product name: ")
        previous = float(input("Enter previous price: "))
        current = float(input("Enter current price: "))

        if previous <= 0 or current < 0:
            print("Invalid price")
            continue

        difference = current - previous
        percentage = (difference / previous) * 100
```

```
        print("\nProduct name:", name)
        print("Previous price: Rs.", previous)
        print("Current price: Rs.", current)
        print("Price difference: Rs.", round(difference, 2))
        print("Percentage change:", round(percentage, 2), "%")

        if difference > 0:
            print("Status: Price increased")
        elif difference < 0:
            print("Status: Price decreased")
        else:
            print("Status: Price unchanged")

print("\nAnalysis completed.")
```

8. Test Cases

Test case	Product	Previous price	Current price	Expected result
1	Rice	₹50	₹60	Increased by ₹10 (20%)
2	Milk	₹40	₹36	Decreased by ₹4 (-10%)
3	Sugar	₹45	₹45	Unchanged (0%)
4	Oil	₹100	₹125	Increased by ₹25 (25%)
5	Soap	₹30	₹0	Decreased by ₹30 (-100%)
6	Biscuits	₹0	₹20	Invalid price

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe c
sers/Velag/OneDrive/Desktop/Python/Activity4.py
MULTI-PRODUCT PRICE FLUCTUATION ANALYSIS
Enter the number of products: 3

Product 1
Enter product name: Rice
Enter previous price: 50
Enter current price: 60

Product name: Rice
Previous price: Rs. 50.0
Current price: Rs. 60.0
Price difference: Rs. 10.0
Percentage change: 20.0 %
Status: Price increased

Product 2
Enter product name: milk
Enter previous price: 40
Enter current price: 36

Product name: milk
Previous price: Rs. 40.0
Current price: Rs. 36.0
Price difference: Rs. -4.0
Percentage change: -10.0 %
Status: Price decreased

Product 3
Enter product name: Sugar
Enter previous price: 45
Enter current price: 45

Product name: Sugar
Previous price: Rs. 45.0
Current price: Rs. 45.0
Price difference: Rs. 0.0
Percentage change: 0.0 %
```

10. Applications

- Supermarkets: To track changes in the prices of products.

- Retail shops: To compare old and new selling prices.

- Customers: To understand whether product prices have increased or decreased.

- Business management: To maintain simple price records for different products.

- Market analysis: To observe price changes over a selected period.

11. Conclusion

The Multi-Product Price Fluctuation Analysis program compares the previous and current prices of multiple products. It calculates the price difference and percentage change and identifies whether each product's price has increased, decreased, or remained unchanged. It also checks for invalid price values. This project helps students understand Python loops, variables, lists of operations across multiple inputs, arithmetic calculations, and conditional statements. It demonstrates how Python can be used to analyze product prices in real-life situations.

5.Student Feedback Sentiment Analysis

1. Problem Statement

Students often provide feedback about teaching methods, classroom facilities, examinations, and college activities. Reading and analyzing a large number of feedback responses manually can take time. The Student Feedback Sentiment Analysis system is a simple Python program that accepts feedback from students and identifies its sentiment using predefined positive and negative words. It calculates the sentiment score and classifies the feedback as positive, negative, or neutral.

2. Objectives

- To accept feedback entered by a student.

- To identify positive and negative words in the feedback.

- To calculate a sentiment score.

- To classify feedback as positive, negative, or neutral.

- To understand Python strings, lists, loops, and conditional statements.

3. Assumptions

1. Feedback is entered as a text sentence.

2. The program uses predefined lists of positive and negative words.

3. Each matching positive word adds one point to the score.

4. Each matching negative word subtracts one point from the score.

5. A score greater than zero indicates positive feedback.

6. A score less than zero indicates negative feedback.

7. A score equal to zero indicates neutral feedback.

8. The program uses basic keyword matching and may not understand sarcasm or the full meaning of every sentence.

4. Algorithm

1. Start the program.

2. Display the title of the Student Feedback Sentiment Analysis system.

3. Enter the student's feedback.

4. Convert the feedback into lowercase.

5. Create lists of positive and negative words.

6. Split the feedback into individual words.

7. Initialize the sentiment score to zero.

8. Check each word in the feedback.

9. If a word is positive, increase the score by one.

10. If a word is negative, decrease the score by one.

11. Repeat until all words have been checked.

12. If the score is greater than zero, classify the feedback as Positive.

13. If the score is less than zero, classify the feedback as Negative.

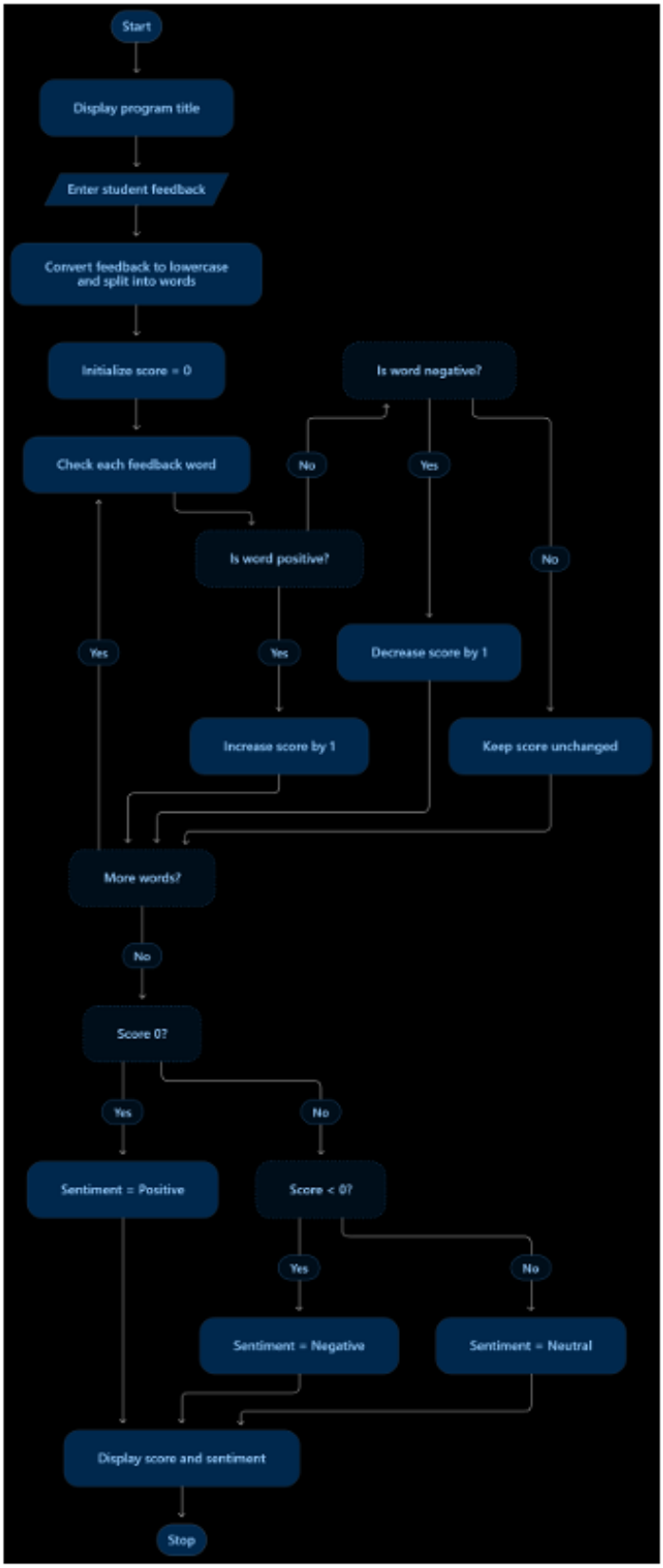
14. Otherwise, classify the feedback as Neutral.

15. Display the sentiment score and result.

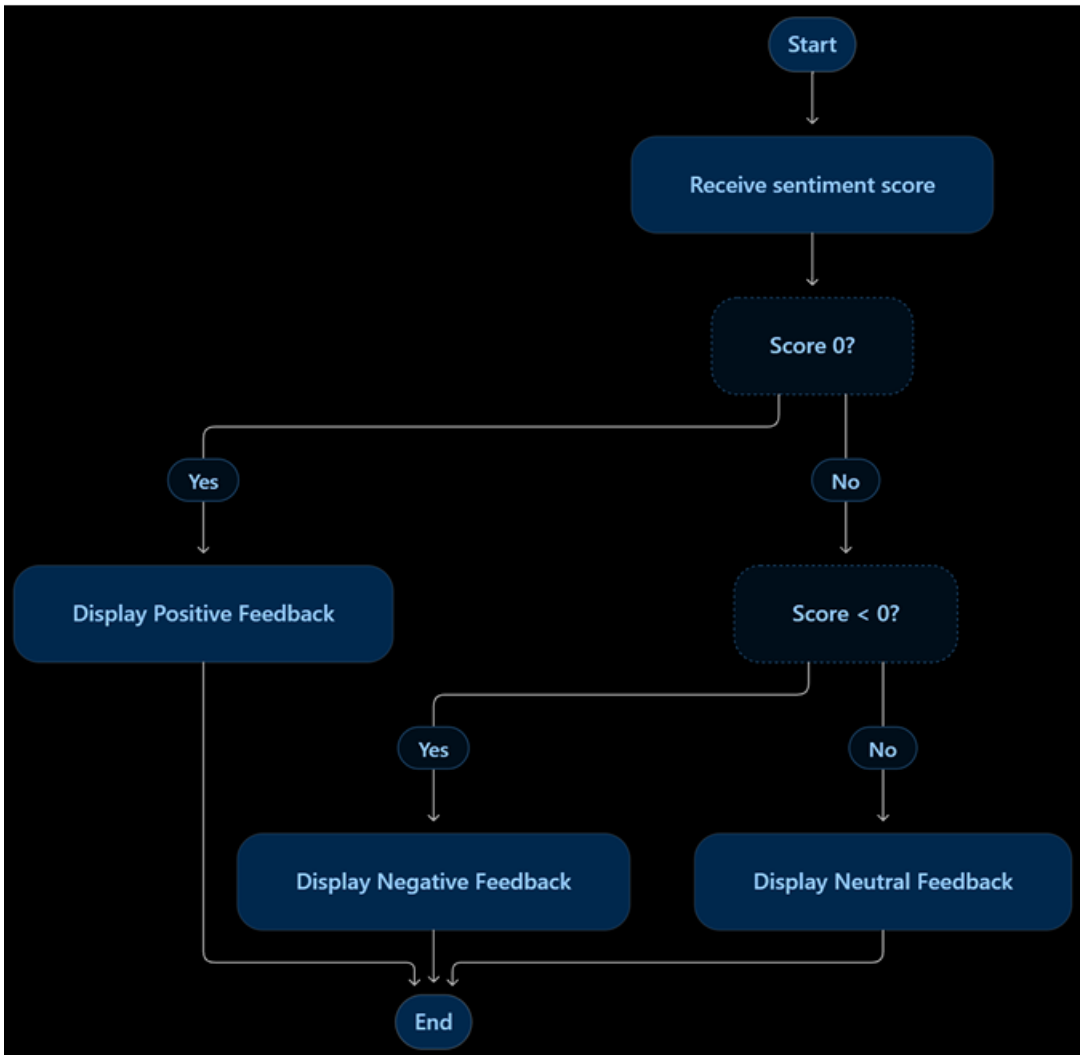
16. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Sentiment classification



6. Pseudocode

```
BEGIN

  DISPLAY "STUDENT FEEDBACK SENTIMENT ANALYSIS"

  INPUT feedback
  CONVERT feedback TO lowercase
  SPLIT feedback INTO words

  positive_words = ["good", "excellent", "helpful",
                    "interesting", "great", "best"]
```

```
negative_words = ["bad", "poor", "boring",
                  "difficult", "worst", "waste"]

score = 0

FOR each word IN words

   REMOVE punctuation from word

   IF word IN positive_words THEN
      score = score + 1

   ELSE IF word IN negative_words THEN
      score = score - 1

   END IF

END FOR

IF score > 0 THEN
   sentiment = "Positive"
ELSE IF score < 0 THEN
   sentiment = "Negative"
ELSE
   sentiment = "Neutral"
END IF
```

DISPLAY score

DISPLAY sentiment

END

7. Python Source Code

```
#Student Feedback Sentiment Analysis

print("STUDENT FEEDBACK SENTIMENT ANALYSIS")

feedback = input("Enter student feedback: ")
feedback = feedback.lower()
words = feedback.split()

positive_words = [
    "good", "excellent", "helpful",
    "interesting", "great", "best"
]

negative_words = [
    "bad", "poor", "boring",
    "difficult", "worst", "waste"
]

score = 0
```

```
for word in words:
    word = word.strip(".,!?")

    if word in positive_words:
        score += 1
    elif word in negative_words:
        score -= 1

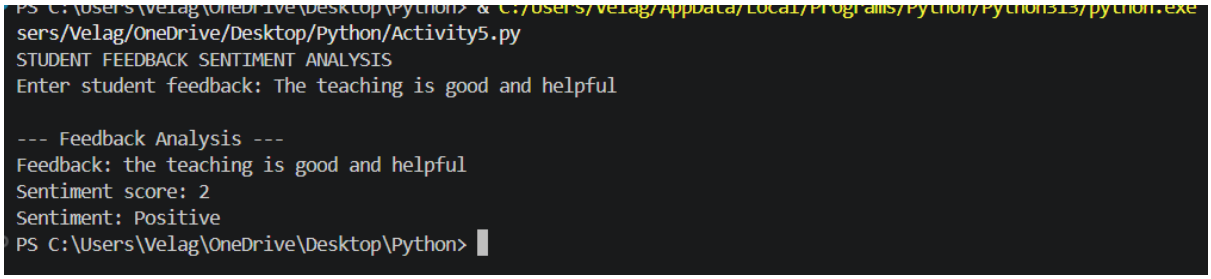
print("\n--- Feedback Analysis ---")
print("Feedback:", feedback)
print("Sentiment score:", score)

if score > 0:
    print("Sentiment: Positive")
elif score < 0:
    print("Sentiment: Negative")
else:
    print("Sentiment: Neutral")
```

8. Test Cases

Test case	Student feedback	Expected score	Expected sentiment
1	The teaching is good and helpful	+2	Positive
2	The class is boring and bad	-2	Negative
3	The classroom is large	0	Neutral
4	Excellent teaching and great explanation	+2	Positive
5	The subject is difficult and boring	-2	Negative
6	Good teaching but boring class	0	Neutral

9. Sample Output



10. Applications

- Colleges: To analyze student opinions about teaching methods.

- Classrooms: To understand students' learning experiences.

- Course evaluation: To identify positive and negative opinions about subjects.

- Educational institutions: To review feedback about facilities and activities.

- Academic improvement: To identify areas that may need improvement.

11. Conclusion

The Student Feedback Sentiment Analysis program analyzes student feedback using predefined positive and negative words. It calculates a sentiment score and classifies the feedback as positive, negative, or neutral. This project helps students understand Python strings, lists, loops, and conditional statements. It also demonstrates how simple programming techniques can be used to analyze text and understand student opinions.

Important limitation: This is a basic keyword-based program. It may not correctly understand sentences involving sarcasm, negation, or context. For example, the sentence "not good" may still receive a positive score because the program does not understand the word "not."

6. Traffic Violation Penalty Calculator

1. Problem Statement

Traffic rules help maintain road safety and prevent accidents. When a person violates traffic rules, a penalty may be imposed depending on the type of violation. The Traffic Violation Penalty Calculator is a simple Python program that identifies a traffic violation and calculates the corresponding penalty using predefined fine amounts. It displays the violation type and the total penalty to help understand the use of conditional statements in Python.

Note: The fine amounts used in this project are hypothetical examples for learning Python, not official traffic fines.

2. Objectives

- To identify different types of traffic violations.

- To assign a sample penalty to each violation.

- To calculate the total penalty amount.

- To display an appropriate message for valid and invalid violation types.

- To understand Python variables, input and output statements, and conditional statements.

3. Assumptions

1. The user selects one traffic violation from a predefined list.

2. Each violation has a fixed sample penalty amount.

3. The penalty amounts are for educational purposes only.

4. The program calculates the penalty for one violation at a time.

5. An invalid violation choice displays an error message.

4. Algorithm

1. Start the program.

2. Display the title of the Traffic Violation Penalty Calculator.

3. Display the list of traffic violations and their sample penalties.

4. Ask the user to enter a violation choice.

5. Check the selected choice.

6. If the choice is 1, assign ₹1,000 for overspeeding.

7. If the choice is 2, assign ₹500 for riding without a helmet.

8. If the choice is 3, assign ₹1,000 for jumping a red signal.

9. If the choice is 4, assign ₹500 for not wearing a seat belt.

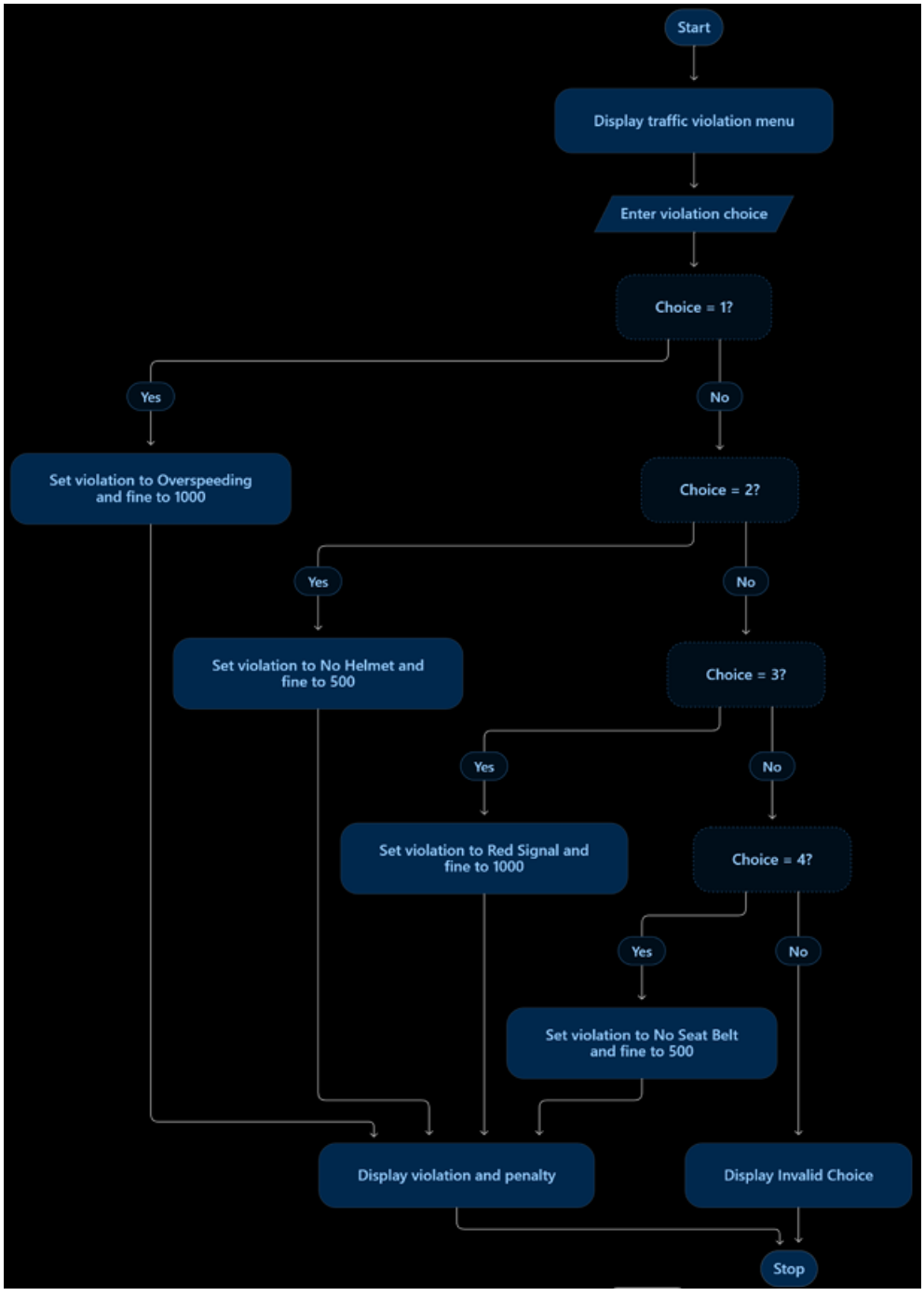
10. If the choice is invalid, display an error message.

11. Otherwise, display the violation type and penalty amount.

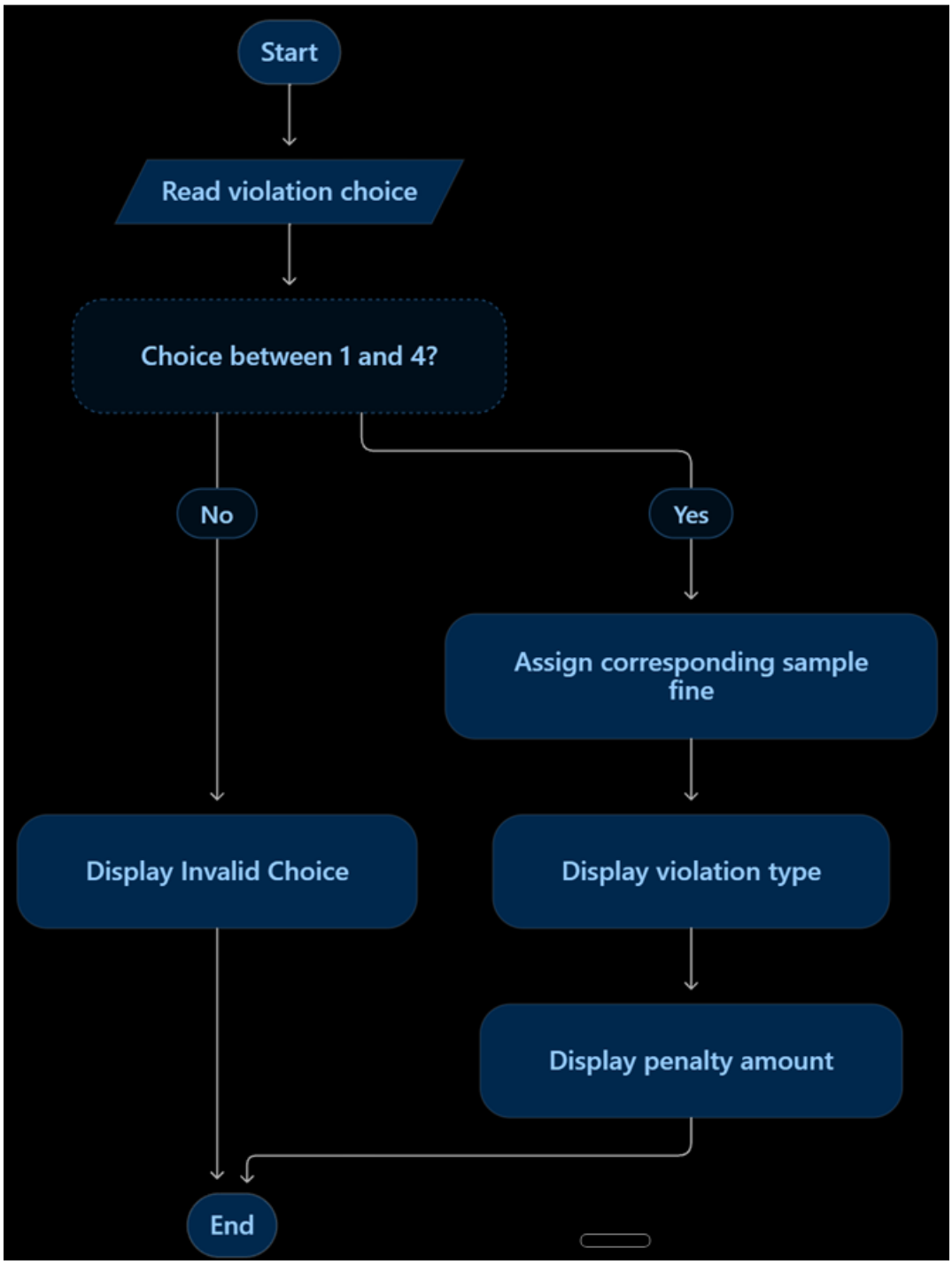
12. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Penalty calculation and validation



6. Pseudocode

BEGIN

DISPLAY "TRAFFIC VIOLATION PENALTY CALCULATOR"

DISPLAY "1. Overspeeding - Rs. 1000"

```
DISPLAY "2. Riding without helmet - Rs. 500"
DISPLAY "3. Jumping red signal - Rs. 1000"
DISPLAY "4. Not wearing seat belt - Rs. 500"

INPUT choice

IF choice == 1 THEN
   violation = "Overspeeding"
   fine = 1000

ELSE IF choice == 2 THEN
   violation = "Riding without helmet"
   fine = 500

ELSE IF choice == 3 THEN
   violation = "Jumping red signal"
   fine = 1000

ELSE IF choice == 4 THEN
   violation = "Not wearing seat belt"
   fine = 500

ELSE
   DISPLAY "Invalid choice"
END IF

IF choice is valid THEN
   DISPLAY violation
```

DISPLAY fine

END IF

END

7. Python Source Code

```
print("TRAFFIC VIOLATION PENALTY CALCULATOR")

print("\n1. Overspeeding - Rs. 1000")
print("2. Riding without helmet - Rs. 500")
print("3. Jumping red signal - Rs. 1000")
print("4. Not wearing seat belt - Rs. 500")

choice = int(input("\nEnter your violation choice (1-4): "))

if choice == 1:
    violation = "Overspeeding"
    fine = 1000

elif choice == 2:
    violation = "Riding without helmet"
    fine = 500

elif choice == 3:
    violation = "Jumping red signal"
    fine = 1000
```

```
elif choice == 4:
    violation = "Not wearing seat belt"
    fine = 500

else:
    violation = ""
    fine = 0
    print("Invalid choice. Please select from 1 to 4.")

if choice in [1, 2, 3, 4]:
    print("\n--- Penalty Details ---")
    print("Violation:", violation)
    print("Penalty amount: Rs.", fine)
```

8. Test Cases

| Test case | Input choice | Expected result |

| --- | --- | --- |

| 1 | 1 | Overspeeding — ₹1,000 |

| 2 | 2 | Riding without helmet — ₹500 |

| 3 | 3 | Jumping red signal — ₹1,000 |

| 4 | 4 | Not wearing seat belt — ₹500 |

| 5 | 5 | Invalid choice |

| 6 | 0 | Invalid choice |

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe
lag/OneDrive/Desktop/Python/Activity 6.py"
TRAFFIC VIOLATION PENALTY CALCULATOR

1. Overspeeding - Rs. 1000
2. Riding without helmet - Rs. 500
3. Jumping red signal - Rs. 1000
4. Not wearing seat belt - Rs. 500

Enter your violation choice (1-4):
```

10. Applications

- Road safety education: Helps students understand common traffic violations.

- Learning Python: Demonstrates the use of conditional statements.

- Traffic awareness programs: Explains how different violations can have different penalties.

- Basic penalty systems: Demonstrates how a menu-based program can assign and display a penalty.

11. Conclusion

The Traffic Violation Penalty Calculator is a simple Python project that identifies a selected traffic violation and displays its corresponding sample penalty. It also handles invalid choices. Through this project, students learn how to use variables, input and output statements, and `if-elif-else` conditional statements. It demonstrates how Python can be used to create a basic decision-making system for a real-life situation.

7. Multi-Product Shopping Budget Constraint

1. Problem Statement

People often purchase multiple products while shopping, but their total expenses may exceed their available budget. The Multi-Product Shopping Budget Constraint system is a simple Python program that accepts a shopping budget and the details of multiple products, including their prices and quantities. It calculates the total cost of the products and compares it with the available budget. Finally, it displays whether the shopping is within the budget or exceeds it.

2. Objectives

- To accept the shopping budget from the user.

- To record the prices and quantities of multiple products.

- To calculate the total cost of shopping.

- To compare the total cost with the available budget.

- To calculate the remaining budget or the extra amount required.

- To understand Python loops, variables, arithmetic operations, and conditional statements.

3. Assumptions

1. The shopping budget must be greater than zero.

2. The number of products must be greater than zero.

3. Product prices must be greater than zero.

4. Product quantities must be positive whole numbers.

5. All prices are entered in Indian rupees (₹).

6. The total cost is calculated by multiplying each product's price by its quantity.

7. The program compares the total shopping cost with the available budget.

4. Algorithm

1. Start the program.

2. Display the title of the shopping budget analysis.

3. Enter the total shopping budget.

4. Enter the number of products to be purchased.

5. Check whether the budget and number of products are valid.

6. If either value is invalid, display an error message.

7. Otherwise, initialize the total cost to zero.

8. For each product, enter the product name, price, and quantity.

9. Check whether the price and quantity are valid.

10. If the product details are invalid, display an error message and stop the calculation.

11. Otherwise, calculate the product cost by multiplying its price by its quantity.

12. Add the product cost to the total shopping cost.

13. Repeat the process for all products.

14. Compare the total shopping cost with the budget.

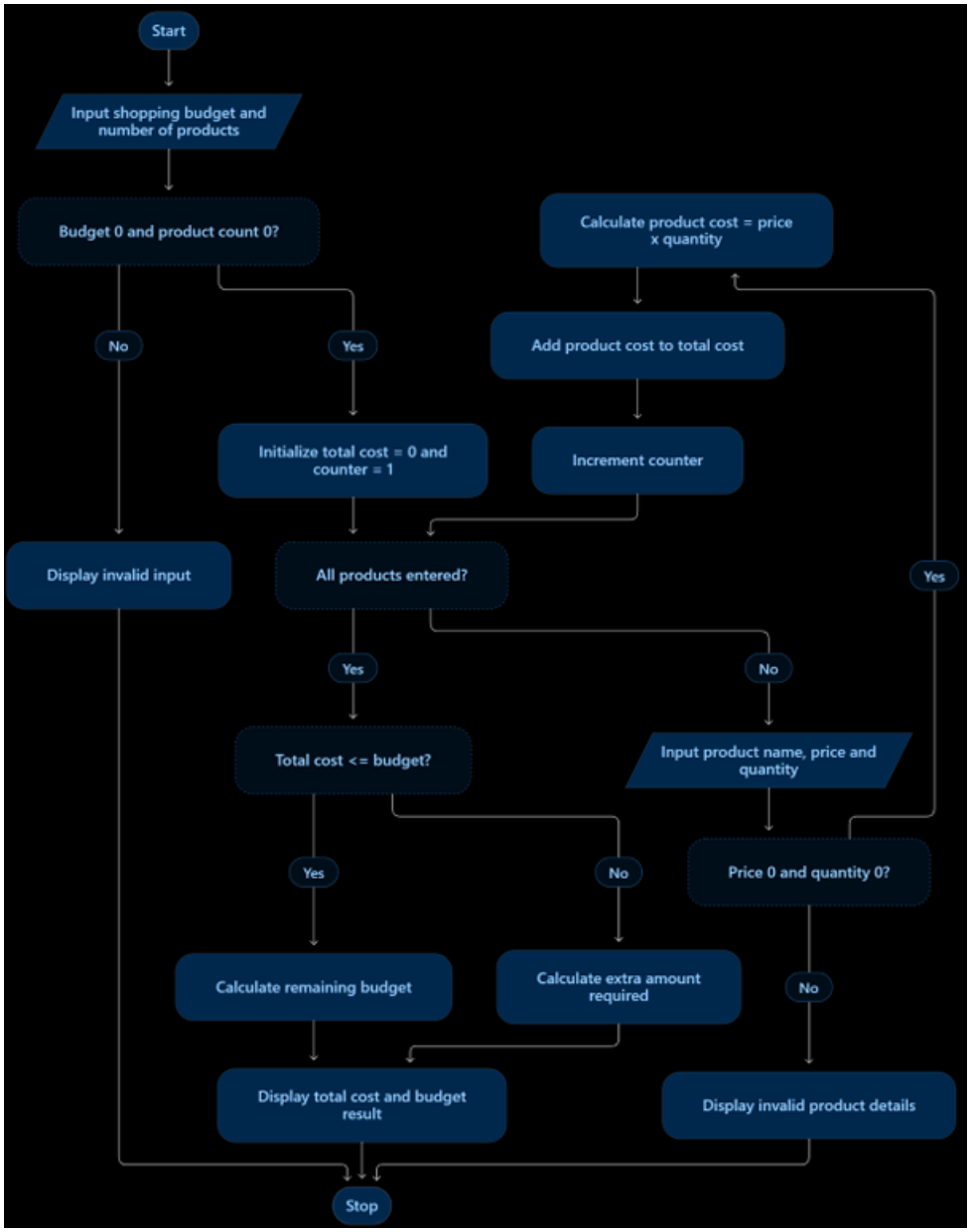
15. If the total cost is less than or equal to the budget, calculate and display the remaining amount.

16. Otherwise, calculate and display the extra amount required.

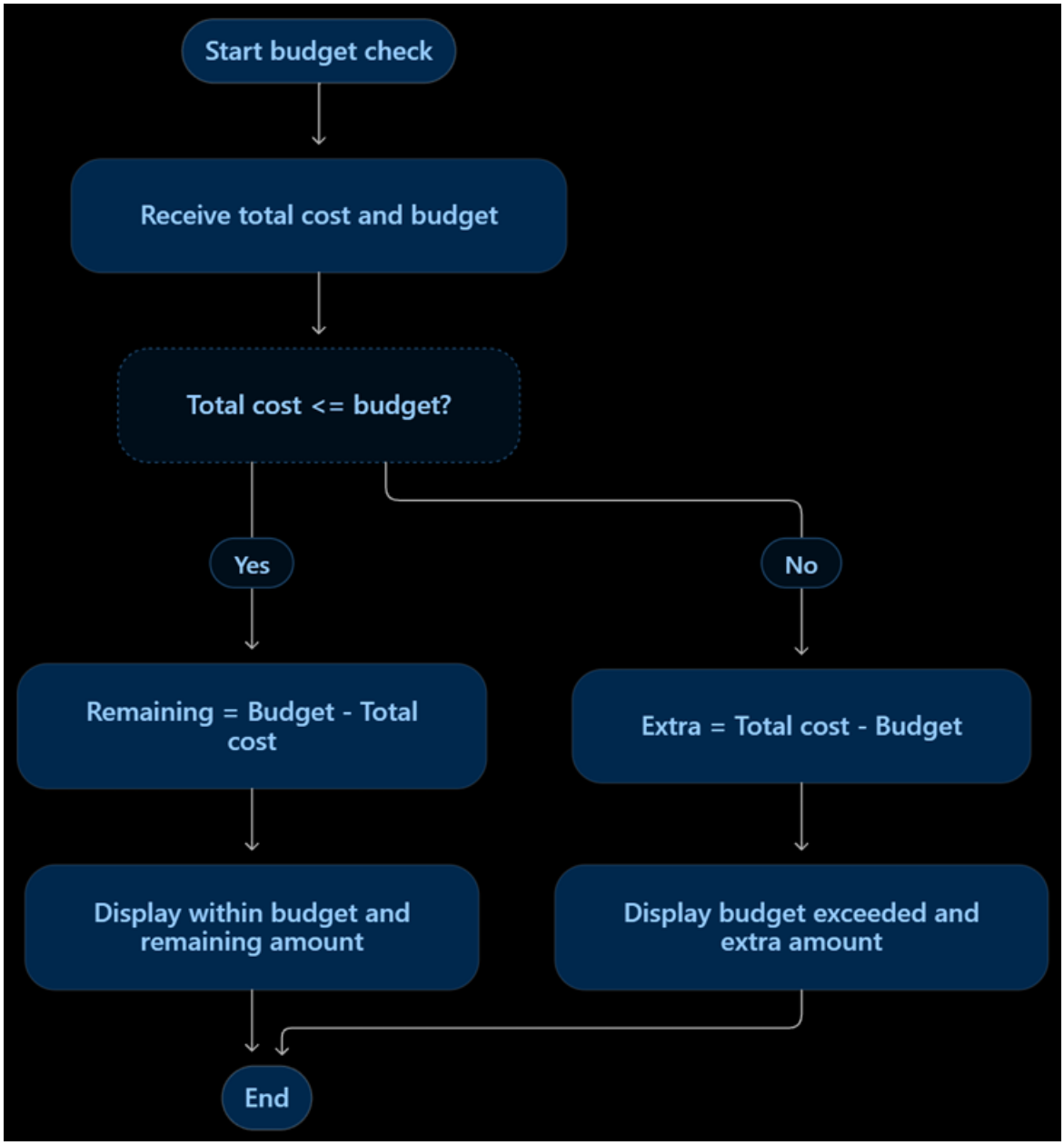
17. Stop the program.

5. Flowcharts

Flowchart 1: Main program



Flowchart 2: Budget checking



6. Pseudocode

```
BEGIN

  DISPLAY "MULTI-PRODUCT SHOPPING BUDGET CONSTRAINT"

  INPUT budget
  INPUT n

  IF budget <= 0 OR n <= 0 THEN
```

```
    DISPLAY "Invalid budget or product count"

ELSE
    total = 0
    valid = TRUE

    FOR each product from 1 to n

        INPUT product_name
        INPUT price
        INPUT quantity

        IF price <= 0 OR quantity <= 0 THEN
            DISPLAY "Invalid product details"
            valid = FALSE
            EXIT LOOP
        ELSE
            cost = price * quantity
            total = total + cost
        END IF

    END FOR

    IF valid == TRUE THEN
        DISPLAY total

        IF total <= budget THEN
            remaining = budget - total
```

DISPLAY "Shopping is within budget"

DISPLAY remaining

ELSE

extra = total - budget

DISPLAY "Budget exceeded"

DISPLAY extra

END IF

END IF

END IF

END

7. Python Source Code

```
print("MULTI-PRODUCT SHOPPING BUDGET CONSTRAINT")

budget = float(input("Enter your shopping budget (Rs.): "))
n = int(input("Enter the number of products: "))

if budget <= 0 or n <= 0:
    print("Invalid budget or number of products.")

else:
    total = 0
    valid = True

    for i in range(n):
        print("\nProduct", i + 1)

        name = input("Enter product name: ")
        price = float(input("Enter product price (Rs.): "))
        quantity = int(input("Enter quantity: "))

```

```
        if price <= 0 or quantity <= 0:
            print("Invalid product details.")
            valid = False
            break

        cost = price * quantity
        total += cost

        print("Product:", name)
        print("Product cost: Rs.", round(cost, 2))

    if valid:
        print("\n--- Shopping Summary ---")
        print("Available budget: Rs.", round(budget, 2))
        print("Total shopping cost: Rs.", round(total, 2))

        if total <= budget:
            remaining = budget - total
            print("Status: Shopping is within budget.")
            print("Remaining budget: Rs.", round(remaining, 2))
```

```
        else:
            extra = total - budget
            print("Status: Budget exceeded.")
            print("Extra amount required: Rs.", round(extra, 2))

    else:
        print("Shopping calculation stopped due to invalid details.")
```

8. Test Cases

Test case	Budget	Products and quantities	Expected result
1	₹1,000	Book: ₹200 × 2; Pen: ₹20 × 5	Total ₹500; remaining ₹500
2	₹500	Bag: ₹400 × 1; Bottle: ₹150 × 1	Total ₹550; extra ₹50 required
3	₹2,000	Shoes: ₹1,200 × 1; Shirt: ₹500 × 1	Total ₹1,700; remaining ₹300
4	₹300	Notebook: ₹50 × 2	Total ₹100; remaining ₹200
5	₹0	Any product	Invalid budget
6	₹1,000	Product price: ₹0	Invalid product details

9. Sample Output

```
PS C:\Users\Velag\OneDrive\Desktop\Python> & C:/Users/Velag/AppData/Local/Programs/Python/Python313/python.exe
lag/OneDrive/Desktop/Python/Activity 7.py"
MULTI-PRODUCT SHOPPING BUDGET CONSTRAINT
Enter your shopping budget (Rs.): 1000
Enter the number of products: 2

Product 1
Enter product name: Book
Enter product price (Rs.): 200
Enter quantity: 2
Product: Book
```

```
Product 2
Enter product name: Pen
Enter product price (Rs.): 20
Enter quantity: 2
Product: Pen
Product cost: Rs. 40.0
```

10. Applications

- Supermarkets: To check whether purchases fit within a shopping budget.

- Household shopping: To manage expenses while purchasing groceries and other items.

- Stationery shops: To calculate the cost of purchasing multiple items.

- Students: To plan purchases within a limited budget.

- Budget management: To identify extra expenses and calculate the remaining amount.

11. Conclusion

The Multi-Product Shopping Budget Constraint program calculates the total cost of multiple products and compares it with the available shopping budget. It displays the remaining budget when the purchase is affordable or the extra amount required when the budget is exceeded. It also checks for invalid product details. Through this project, students learn Python variables, loops, arithmetic operations, and conditional statements. This project demonstrates how Python can help manage shopping expenses in everyday life.

8. Student Performance Trend Analysis

1. Introduction

Student Performance Trend Analysis is a simple Python project used to analyze a student's marks in different tests. It compares the marks and identifies whether the student's performance is improving, decreasing, or remaining the same.

The program takes the marks of multiple tests as input, calculates the average marks, and compares the first test marks with the last test marks. It then displays the performance trend and average marks.

2. Problem Statement

To develop a Python program that accepts a student's marks in multiple tests, calculates the average marks, and identifies the student's performance trend based on the marks obtained in the first and last tests.

3. Objectives

- To collect marks from multiple tests.

- To calculate the average marks.

- To compare the first and last test marks.

- To identify whether performance is improving, decreasing, or unchanged.

- To display a simple performance report.

4. Assumptions

- Marks are entered out of 100.

- At least two tests are required to analyze the trend.

- Marks must be between 0 and 100.

- The program compares the first test and the last test only.

- The program uses basic Python concepts such as loops, conditions, and lists.

5. Algorithm

1. Start the program.

2. Display the project title.

3. Enter the number of tests.

4. If the number of tests is less than 2, display an invalid input message.

5. Otherwise, initialize an empty list to store marks.

6. Enter marks for each test.

7. Check whether each mark is between 0 and 100.

8. If marks are invalid, display an error message and stop the calculation.

9. If all marks are valid, calculate the average marks.

10. Compare the first test marks with the last test marks.

11. If the last marks are greater, display “Performance is improving.”

12. If the last marks are lower, display “Performance is decreasing.”

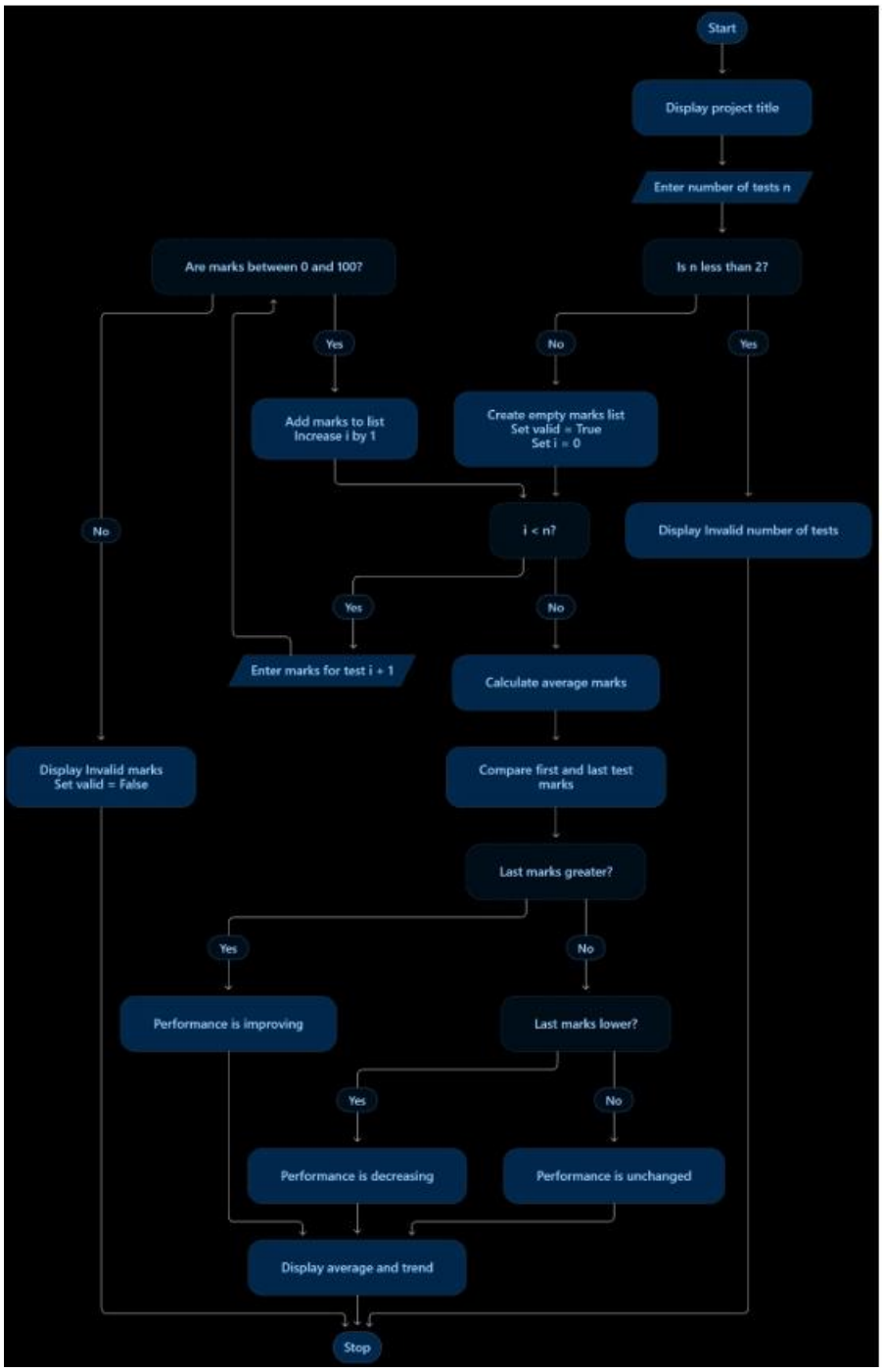
13. Otherwise, display “Performance is unchanged.”

14. Display the average marks and performance trend.

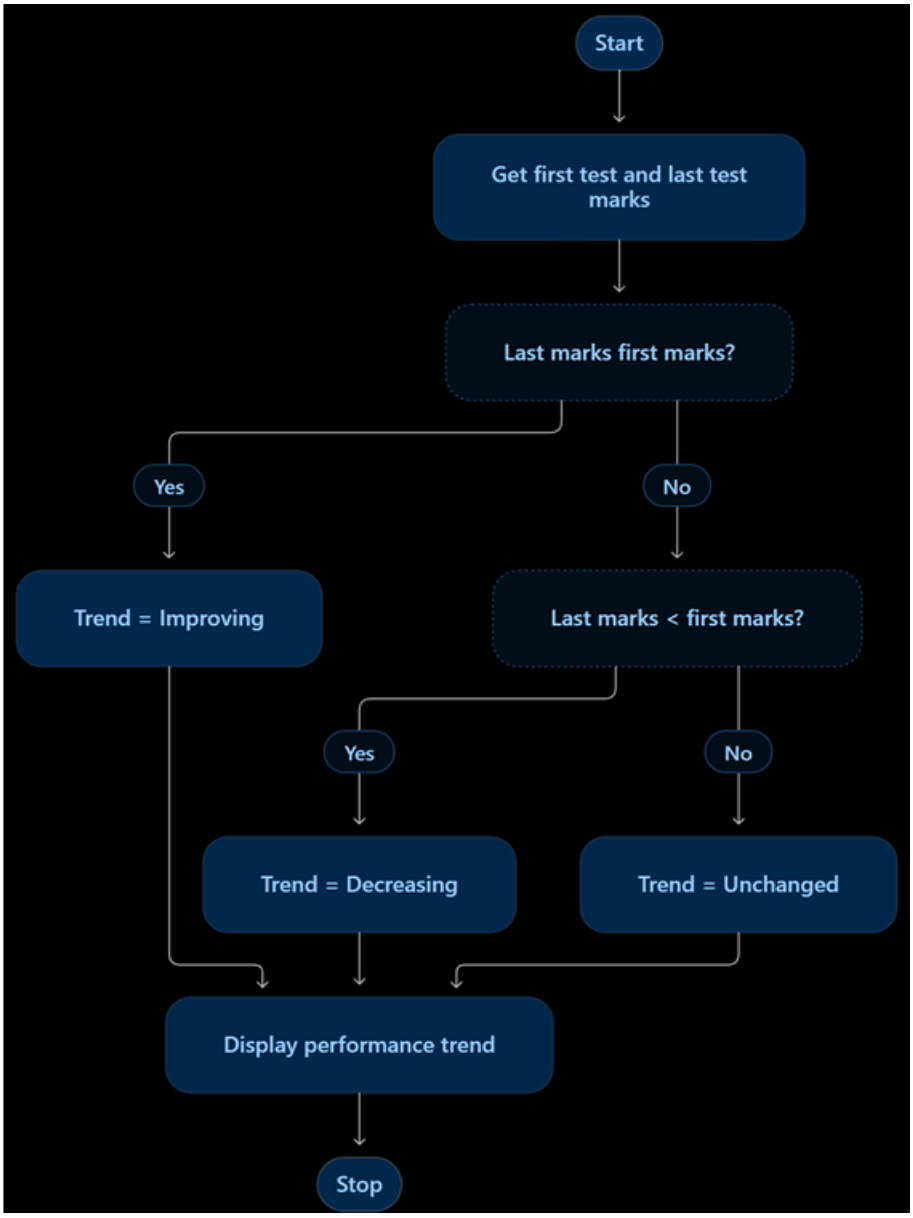
15. Stop the program.

6. Flowchart

Flowchart 1: Main program


Start
Display project title
Enter number of tests n
Is n less than 2?
Yes
No
Yes
Display Invalid number of tests
Are marks between 0 and 100?
Yes
No
Add marks to list
Increase i by 1
Create empty marks list
Set valid = True
Set i = 0
i < n?
Yes
No
Enter marks for test i + 1
Calculate average marks
Compare first and last test marks
Last marks greater?
Yes
No
Performance is improving
Last marks lower?
Yes
No
Performance is decreasing
Performance is unchanged
Display average and trend
Stop
No
Display Invalid marks
Set valid = False

Flowchart 2: Performance trend analysis



7. Pseudocode

START

DISPLAY "STUDENT PERFORMANCE TREND ANALYSIS"

INPUT n

IF n < 2 THEN

  DISPLAY "Invalid number of tests"

ELSE

```
CREATE empty list marks
SET valid = True

FOR each test from 1 to n
  INPUT mark

  IF mark < 0 OR mark > 100 THEN
    DISPLAY "Invalid marks"
    SET valid = False
    STOP LOOP
  ELSE
    ADD mark to marks
  END IF
END FOR

IF valid = True THEN
  CALCULATE average marks

  IF last marks > first marks THEN
    DISPLAY "Performance is improving"
  ELSE IF last marks < first marks THEN
    DISPLAY "Performance is decreasing"
  ELSE
    DISPLAY "Performance is unchanged"
  END IF

  DISPLAY average marks
  DISPLAY performance trend
```

END IF

END IF

STOP

8. Python Source Code

```
print("STUDENT PERFORMANCE TREND ANALYSIS")

n = int(input("Enter the number of tests: "))

if n < 2:
    print("Invalid number of tests. Enter at least 2 tests.")
else:
    marks = []
    valid = True

    for i in range(n):
        mark = float(input("Enter marks for test " + str(i + 1) + ": "))

        if mark < 0 or mark > 100:
            print("Invalid marks. Marks must be between 0 and 100.")
            valid = False
            break
        else:
            marks.append(mark)

    if valid:
        total = sum(marks)
        average = total / n

        first = marks[0]
        last = marks[-1]

        print("\n--- Performance Report ---")
        print("First test marks:", first)
        print("Last test marks:", last)
        print("Average marks:", round(average, 2))
```

```
        if last > first:
            print("Performance trend: Improving")
        elif last < first:
            print("Performance trend: Decreasing")
        else:
            print("Performance trend: Unchanged")
```

9. Test Cases

| Test case | Input marks | Expected result |

| --- | --- | --- |

| 1 | 50, 60, 75 | Improving |

| 2 | 90, 80, 70 | Decreasing |

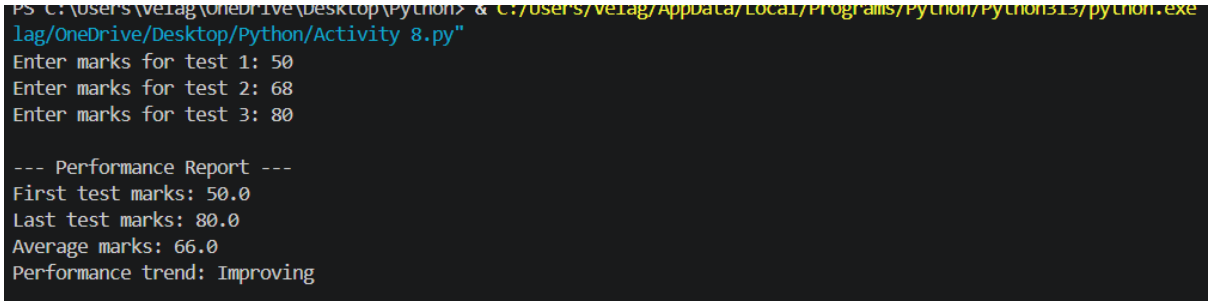
| 3 | 65, 70, 65 | Unchanged |

| 4 | 40, 60 | Improving |

| 5 | 50, 110, 70 | Invalid marks |

| 6 | Only 1 test | Invalid number of tests |

10. Sample Output



11. Applications

- Tracking student performance across tests.

- Helping students understand their academic progress.

- Assisting teachers in identifying performance changes.

- Supporting academic progress reports.

- Helping students identify subjects or tests where they may need more practice.

12. Conclusion

The Student Performance Trend Analysis project shows how Python can be used to analyze students' marks. It calculates average marks and compares the first and last test results to identify the performance trend. This project helps beginners understand lists, loops, conditional statements, and basic calculations in Python.

Important: This program checks only the first and last test marks to identify the trend. It does not guarantee that marks increased or decreased in every test between them.

9. Warehouse Inventory Reconciliation

1. Introduction

Warehouse Inventory Reconciliation is a Python project used to check the quantity of products recorded in a warehouse against the quantity actually available. Sometimes, the recorded stock and the physical stock may be different due to mistakes, damaged products, or missing items.

This program accepts the product name, recorded stock quantity, and actual stock quantity. It compares both quantities and displays whether the stock matches, has a shortage, or has extra items.

2. Problem Statement

To develop a Python program that compares the recorded stock quantity with the actual stock quantity of warehouse products and identifies whether the inventory matches, has a shortage, or contains extra items.

3. Objectives

- To enter product details and stock quantities.

- To compare recorded stock with actual stock.

- To identify stock shortages and extra stock.

- To display the difference between the two quantities.

- To help maintain accurate warehouse inventory records.

4. Assumptions

- Stock quantities are entered as whole numbers.

- Stock quantities cannot be negative.

- The program checks one product at a time.

- Recorded stock means the quantity in the warehouse records.

- Actual stock means the quantity physically counted in the warehouse.

5. Algorithm

1. Start the program.

2. Display the project title.

3. Enter the product name.

4. Enter the recorded stock quantity.

5. Enter the actual stock quantity.

6. Check whether either quantity is negative.

7. If any quantity is negative, display an invalid input message.

8. Otherwise, compare the recorded and actual stock quantities.

9. If both quantities are equal, display “Stock matches.”

10. If actual stock is less than recorded stock, calculate and display the stock shortage.

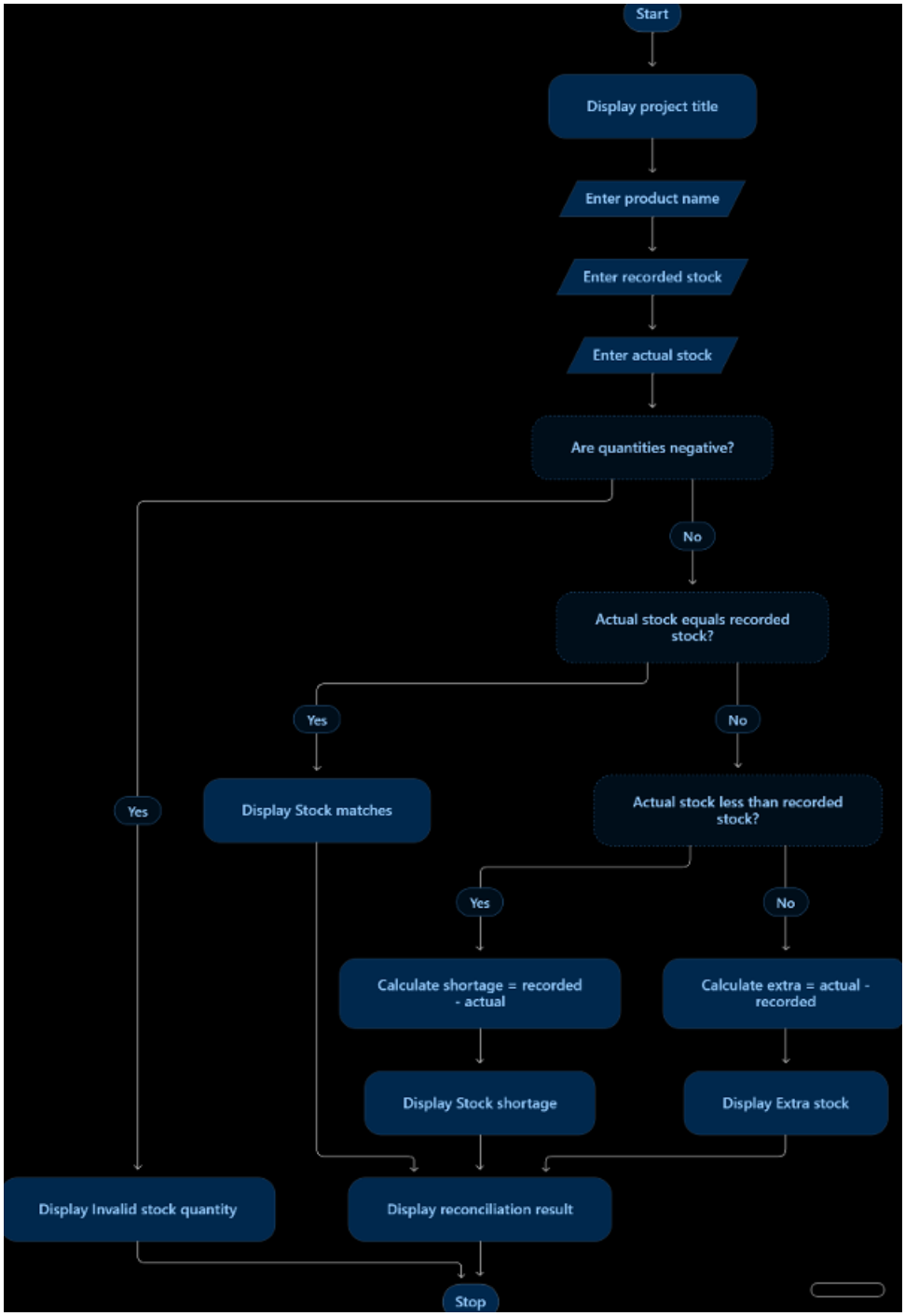
11. If actual stock is greater than recorded stock, calculate and display the extra stock.

12. Display the inventory reconciliation result.

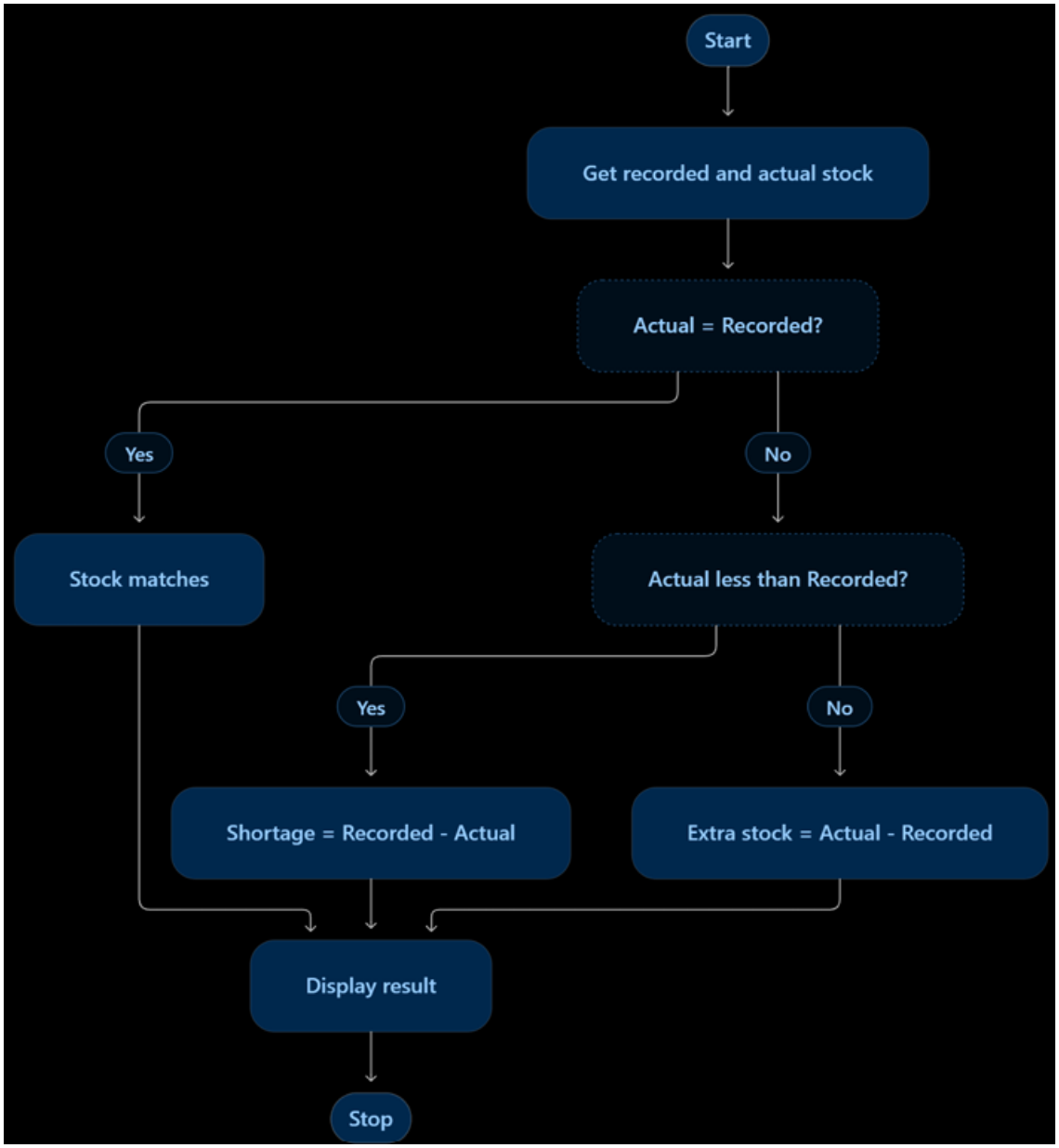
13. Stop the program.

6. Flowchart

Flowchart 1: Complete program



Flowchart 2: Stock comparison



7. Pseudocode

START

DISPLAY "WAREHOUSE INVENTORY RECONCILIATION"

INPUT product_name

INPUT recorded_stock

INPUT actual_stock

```
IF recorded_stock < 0 OR actual_stock < 0 THEN
   DISPLAY "Invalid stock quantity"
ELSE
   IF actual_stock = recorded_stock THEN
      DISPLAY "Stock matches"
      DISPLAY "No difference in stock"

   ELSE IF actual_stock < recorded_stock THEN
      shortage = recorded_stock - actual_stock
      DISPLAY "Stock shortage"
      DISPLAY shortage

   ELSE
      extra = actual_stock - recorded_stock
      DISPLAY "Extra stock"
      DISPLAY extra
   END IF

   DISPLAY product_name
   DISPLAY reconciliation result
END IF

STOP
```

8. Python Source Code

```
print("WAREHOUSE INVENTORY RECONCILIATION")

product = input("Enter product name: ")
recorded = int(input("Enter recorded stock quantity: "))
actual = int(input("Enter actual stock quantity: "))

if recorded < 0 or actual < 0:
    print("Invalid stock quantity. Enter values from 0 onwards.")

else:
    print("\n--- Inventory Report ---")
    print("Product name:", product)
    print("Recorded stock:", recorded)
    print("Actual stock:", actual)

    if actual == recorded:
        print("Status: Stock matches")
        print("Stock difference: 0")

    elif actual < recorded:
        shortage = recorded - actual
        print("Status: Stock shortage")
        print("Missing quantity:", shortage)

    else:
        extra = actual - recorded
        print("Status: Extra stock")
        print("Extra quantity:", extra)

    print("Inventory reconciliation completed.")
```

9. Test Cases

Test case	Recorded stock	Actual stock	Expected result
1	100	100	Stock matches
2	100	80	Shortage of 20
3	100	120	Extra stock of 20
4	50	0	Shortage of 50
5	0	10	Extra stock of 10
6	-10	20	Invalid stock quantity

10. Sample Output

```
lag/OneDrive/Desktop/Python/Activity 9.py"
WAREHOUSE INVENTORY RECONCILIATION
Enter product name: Notebook
Enter recorded stock quantity: 100
Enter actual stock quantity: 85

--- Inventory Report ---
Product name: Notebook
Recorded stock: 100
Actual stock: 85
```

```
Status: Stock shortage
Missing quantity: 15
Inventory reconciliation completed.
PS C:\Users\Velag\OneDrive\Desktop\Python>
```

11. Applications

- Checking stock in warehouses.

- Identifying missing or extra products.

- Maintaining accurate inventory records.

- Supporting stock verification in shops and supermarkets.

- Helping warehouse staff identify differences between recorded and physical stock.

12. Conclusion

The Warehouse Inventory Reconciliation project demonstrates how Python can be used to compare recorded stock with actual stock. It identifies matching stock, shortages, and extra quantities using conditional statements and basic arithmetic. This project helps beginners understand input handling, decision-making, and calculations in Python.

10. Student Email and Identifiers Validation

1. Introduction

Student Email and Identifiers Validation is a simple Python project used to check whether student email addresses and student identifiers are entered in the correct format. These details are commonly used in colleges for student registration, attendance, and communication.

The program accepts a student's email address and student ID. It checks whether the email contains the required symbols and whether the student ID follows the expected format. Finally, it displays whether each detail is valid or invalid.

2. Problem Statement

To develop a Python program that validates a student's email address and student identifier using simple conditions and displays the validation results.

3. Objectives

- To accept a student's email address and student ID.

- To check the basic format of an email address.

- To verify the format of a student identifier.

- To display valid or invalid results.

- To understand Python strings, conditions, and input validation.

4. Assumptions

- The email address must contain `@` and a dot (`.`) after `@`.

- The email address must not contain spaces.

- The student ID must begin with `STU` followed by digits, such as `STU123`.

- The student ID must contain at least one digit after `STU`.

- These are basic project rules, not a complete check of real-world email addresses or official college ID formats.

5. Algorithm

1. Start the program.

2. Display the project title.

3. Enter the student's email address.

4. Enter the student's ID.

5. Check whether the email contains exactly one `@`, a dot after `@`, and no spaces.

6. If all email conditions are satisfied, mark the email as valid; otherwise, mark it as invalid.

7. Check whether the student ID starts with `STU` and is followed by one or more digits.

8. If the ID conditions are satisfied, mark the ID as valid; otherwise, mark it as invalid.

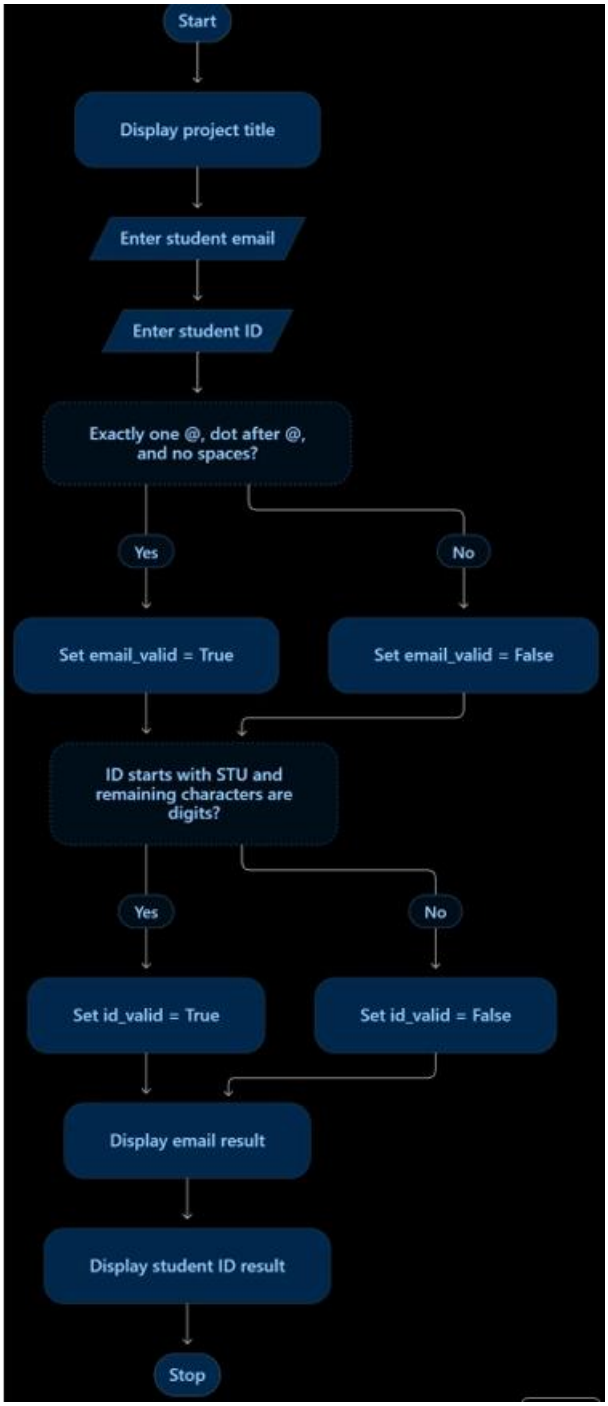
9. Display the email validation result.

10. Display the student ID validation result.

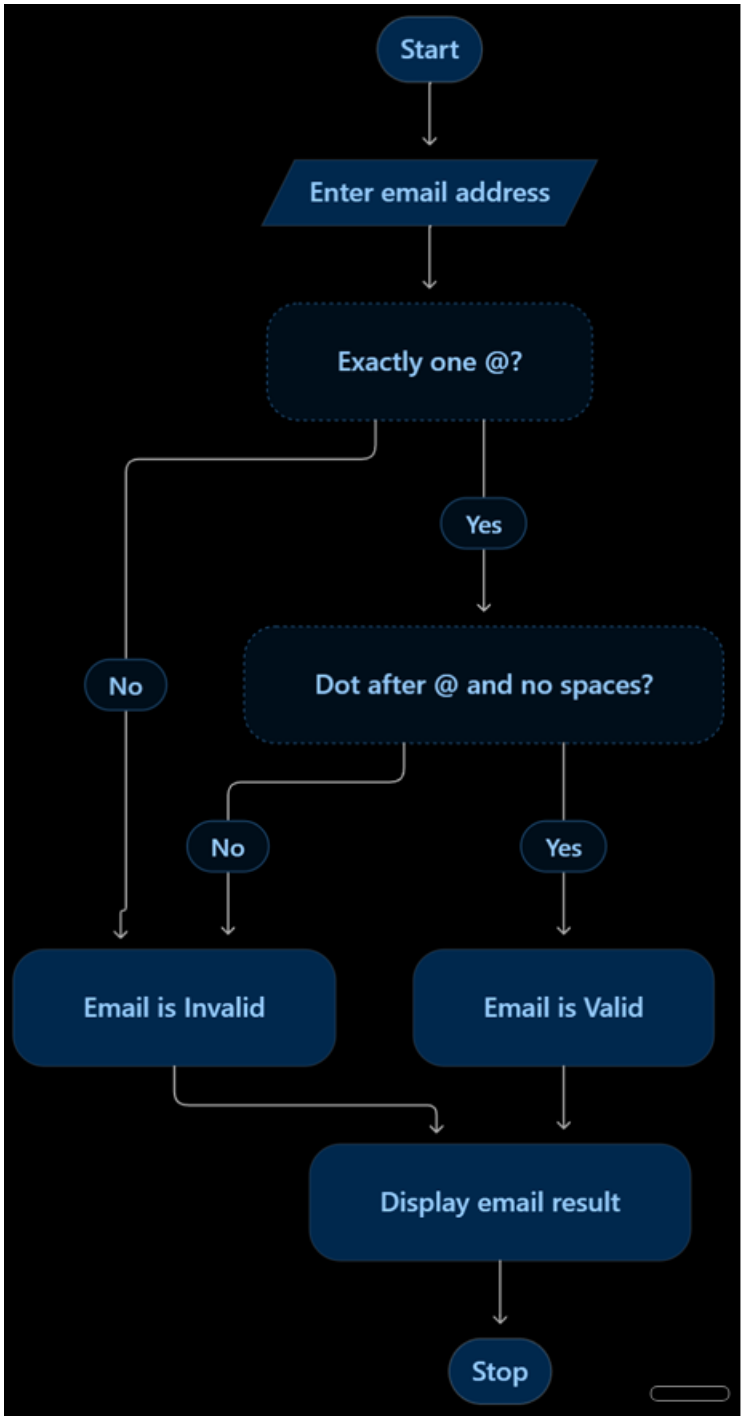
11. Stop the program.

6. Flowchart

Flowchart 1: Complete program



Flowchart 2: Email validation



7. Pseudocode

START

DISPLAY "STUDENT EMAIL AND IDENTIFIERS VALIDATION"

INPUT email

```
INPUT student_id

IF email contains exactly one "@"
  AND dot occurs after "@"
  AND email contains no spaces THEN
   SET email_valid = True
ELSE
   SET email_valid = False
END IF

IF student_id starts with "STU"
  AND remaining characters are digits THEN
   SET id_valid = True
ELSE
   SET id_valid = False
END IF

IF email_valid = True THEN
   DISPLAY "Email is Valid"
ELSE
   DISPLAY "Email is Invalid"
END IF

IF id_valid = True THEN
   DISPLAY "Student ID is Valid"
ELSE
   DISPLAY "Student ID is Invalid"
END IF
```

STOP

8. Python Source Code

```
print("STUDENT EMAIL AND IDENTIFIERS VALIDATION")

email = input("Enter student email: ")
student_id = input("Enter student ID: ")

# Email validation
if (email.count("@") == 1
        and "." in email.split("@")[1]
        and " " not in email):
    email_valid = True
else:
    email_valid = False

# Student ID validation
if student_id.startswith("STU") and student_id[3:].isdigit():
    id_valid = True
else:
    id_valid = False

# Display results
print("\n--- Validation Results ---")

if email_valid:
    print("Email:", email)
    print("Email status: Valid")
else:
    print("Email status: Invalid")

if id_valid:
    print("Student ID:", student_id)
    print("Student ID status: Valid")
else:
    print("Student ID status: Invalid")
```

9. Test Cases

| Test | Email | Student ID | Expected result |

| ---- | ---------- | ---------- | ---------- |

| 1 | `student@gmail.com` | `STU123` | Both valid |

| 2 | `studentgmail.com` | `STU123` | Invalid email |

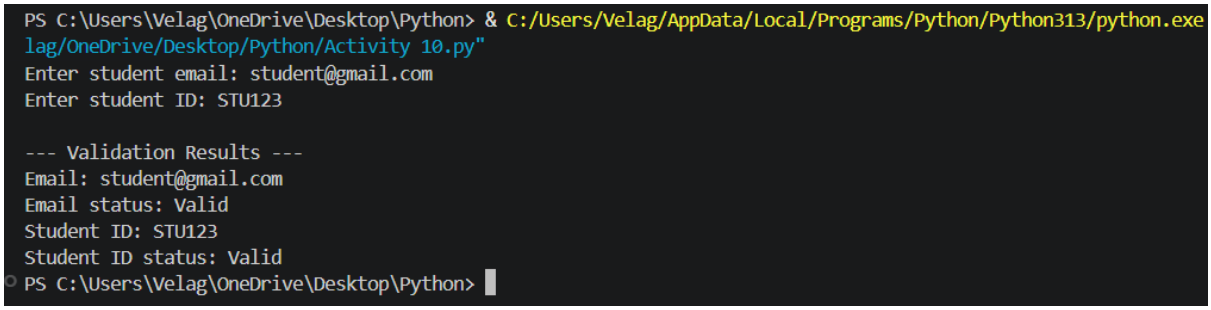
| 3 | `student@gmail.com` | `ABC123` | Invalid ID |

| 4 | `student @gmail.com` | `STU456` | Invalid email |

| 5 | `student@gmail.com` | `STU12A` | Invalid ID |

| 6 | `student@gmail.com` | `STU` | Invalid ID |

10. Sample Output



11. Applications

- Student registration systems.

- College admission forms.

- Student database management.

- Online learning platforms.

- Checking basic email and student ID formats before storing information.

12. Conclusion

The Student Email and Identifiers Validation project demonstrates how Python can check student information using string operations and conditional statements. It validates the basic email format and checks whether the student ID follows the specified pattern. This project helps beginners understand string methods, Boolean variables, and decision-making in Python.